university of groningen

faculty of science and engineering

Contact-Rich Planar Goal-Reaching: Why Asymmetric Self-Play Underperforms Single-Agent Training

Vlad Iftime

university of groningen / faculty of science and engineering

University of Groningen

Contact-Rich Planar Goal-Reaching: Why Asymmetric Self-Play Underperforms Single-Agent Training

Master's Thesis

To fulfill the requirements for the degree of
Master of Science in Artificial Intelligence
at University of Groningen under the supervision of
Hamidreza Mohades Kasaei, PhD (Associate Professor)
and Matthia Sabatelli, PhD (Assistant Professor)

Vlad Iftime (s3426394)

August 24, 2026

Contents

Page

Abstract **5**

1 Introduction **6**
1.1 Contributions . . . 7
1.2 Thesis Outline . . . 8

2 Background and Related Work **9**
2.1 Reinforcement Learning . . . 9
2.1.1 Markov Decision Processes . . . 9
2.1.2 Partial Observability and the POMDP . . . 10
2.1.3 Model-Free and Model-Based Reinforcement Learning . . . 11
2.2 Proximal Policy Optimization . . . 12
2.3 Soft Actor-Critic . . . 12
2.4 Potential-Based Reward Shaping . . . 13
2.5 Curriculum Learning and Automatic Curricula . . . 13
2.6 Asymmetric Self-Play . . . 14
2.7 Goal-Conditioned and Hierarchical Reinforcement Learning . . . 15
2.8 Mechanics and Geometry of Planar Pushing . . . 16
2.9 Positioning of This Work . . . 17

3 Methods **18**
3.1 Simulation Environment . . . 18
3.2 The Planar-Pushing Markov Decision Process . . . 19
3.3 The Object-Relative Push Primitive . . . 20
3.4 Why the Improvement Reward Underperforms . . . 22
3.5 Potential-Based Reward Shaping for Pushing . . . 22
3.6 The SE(2) Unified Distance . . . 24
3.7 The Models . . . 24
3.8 Why PBRS Assists Self-Play . . . 30
3.9 Experiment-Set Overview . . . 31
3.10 Validation Protocol . . . 32
3.11 Training Instrumentation for the Self-Play Mechanism . . . 33

4 Experimental Setup **34**
4.1 Tools and Infrastructure . . . 34
4.2 Training Configuration and the Experiment Set . . . 34
4.3 Validation Protocol . . . 35
4.4 Cross-Environment Protocol . . . 36
4.5 Off-Policy Baseline: SAC with Hindsight Experience Replay . . . 37
4.6 Metrics . . . 38

5 Results **39**
5.1 Baseline . . . 39
5.2 Research Question 1: ASP performance under the single-GPU budget . . . 40
5.3 Research Question 2: Setup dimensions . . . 41
5.3.1 Goal Coupling . . . 41
5.3.2 Reward Formulation . . . 42
5.3.3 Compute Scale . . . 42
5.3.4 Imitation Signal . . . 42
5.3.5 Goal Representation . . . 42
5.4 Research Question 3: Mechanism . . . 43
5.5 Cross-Environment . . . 44
5.6 Qualitative Behaviour . . . 44

6 Discussion and Conclusion **46**
6.1 Interpretation . . . 46
6.2 Relation to Prior Work . . . 46
6.3 Contributions Revisited . . . 47
6.4 Limitations . . . 48
6.5 Future Work . . . 48
6.6 Conclusion . . . 49

Bibliography **50**

Appendices **54**
A Push Mechanics: Limit Surface and Characteristic Length . . . 54
B Training Curves . . . 54
C PBRS Parameter Sensitivity . . . 55
D Validation Scene Descriptions . . . 55
E cuRobo IK Pipeline and Workspace . . . 55
F Observation Space Specifications . . . 55
G Confounded Ablation Disaggregation . . . 56
H Training Budget Comparison . . . 56
I Secondary Variants . . . 56

Abstract

On-policy reinforcement learning for contact-rich robotic manipulation requires extensive interaction. This thesis studies one task: planar pushing of a T-block to a target pose by a simulated UR5e arm in NVIDIA Isaac Lab. Pushing mechanics couple translation and rotation, so a goal that specifies a position and an orientation cannot be split into independent sub-problems.

Asymmetric Self-Play (ASP) is an automatic curriculum that succeeds on goal-reaching problems elsewhere, yet on the pushing task it transfers none of its within-distribution competence to held-out scenes. A plain single-agent policy with a potential-based reward reaches 79.3 % held-out scene success (± 3.5 pp, five seeds). The self-play subjects solve 0.0 % on the T-block and 7.3 % on a rotationally symmetric disc. The mechanism is a generalisation failure compounded by the coupled multi-objective gate.

A within-distribution evaluation shows the confinement: on goals the goal-proposing agent itself produces, the self-play policy solves 61.4 % of disc scenes and 11.7 % of T-block scenes, against 7.3 % and 0.0 % of held-out scenes. The imitation term is suppressed in discrete macro-actions (clip fraction 0.000); an eightfold larger batch leaves the result unchanged. ASP succeeds in the settings prior work used. The thesis explains why it does not generalise under a single Graphical Processing Unit budget. For this class of task, principled reward design produces a larger improvement than structured training.

1 Introduction

Service robots are expected to become part of everyday life, assisting people at home and at work with ordinary manipulation tasks. Meeting that expectation requires a robot to handle objects it has never seen in cluttered and unstructured surroundings [1]. Bin picking is the canonical instance of this challenge: a robot must retrieve target items from a dense, disordered container, where objects rest against one another and few are graspable as they lie. Traditional pipelines that assume known object models and a clear grasp pose become brittle here, because the required information is often occluded or simply unavailable. Learning-based methods, and deep Reinforcement Learning (RL) in particular, have therefore become the dominant approach to manipulation in clutter [1].

A recurring insight in this literature is that grasping alone is not enough. When objects are packed together, the robot must first rearrange them, and non-prehensile actions such as pushing create the space and the poses that make a later grasp feasible. Push-grasp synergy, in which pushing and grasping are learned jointly, improves picking success in dense clutter beyond what grasping achieves on its own [2, 3, 4]. This positions pushing as a foundational pre-grasp skill for service robots: the ability to move an object into a target pose is what turns an ungraspable configuration into a graspable one. This thesis studies that skill in isolation, framed as a goal-reaching problem in which the agent must drive the object to a target pose. In doing so it works toward the same broad goal as the clutter-manipulation literature it builds on, namely competent, general manipulation in unstructured environments.

Progress on this goal is limited less by what RL can express than by what it costs to train. On-policy algorithms such as Proximal Policy Optimization (PPO) are stable and widely used [5], but they discard each batch of experience after a single update. Their sample requirement is therefore high, and it grows with the difficulty of the reward signal. When the reward is sparse or poorly shaped, most of the interaction budget yields little useful gradient, and the number of parallel environments needed for a stable update becomes the dominant practical constraint. Reducing this sample cost, and understanding which methods reduce it, is therefore a central concern for learning manipulation skills of this kind.

Two families of techniques aim to reduce this constraint, and they act on different parts of the problem. The first is reward shaping, which adds an auxiliary signal that directs the agent toward useful behaviour without changing what the task ultimately rewards. Potential-Based Reward Shaping (PBRS) is the principled form of this idea [6]: its shaping term is the difference of a state potential, which leaves the optimal policy unchanged. The second family modifies the training process rather than the reward. Curriculum learning orders tasks from easy to hard [7], and its automatic variants derive that order from the agent's own competence. Asymmetric Self-Play (ASP) is the most prominent automatic form [8, 9]: one agent proposes goals while a second learns to reach them, so the goal distribution adapts to the learner without human specification. Both families are well motivated. Reward shaping, in the potential-based form used here, produces a single-agent policy that solves the task, and serves as the baseline of this study. ASP is the method under scrutiny: it has succeeded on goal-reaching problems elsewhere, including robotic manipulation [9] and reversible control tasks [8], yet on the task studied here it underperforms the single-agent baseline. That contrast is what this this thesis addresses.

The task studied is planar pushing, reduced to a single object and framed as a goal-reaching problem so that the learning question can be isolated. A robot must slide the object across a surface to a target pose. Pushing suits this study for a specific mechanical reason. Under

the quasi-static assumption, a pushed object's motion follows its friction limit surface, and the resulting twist couples translation and rotation [10, 11]. Almost every push therefore changes both the position and the orientation of the object. A goal that fixes both a target position and orientation is thus multi-objective, and the two objectives cannot be optimised independently. This coupling makes planar pushing a compact and demanding evaluation task: the reward must give an accurate gradient for two interacting quantities, and any curriculum or self-play mechanism must satisfy both criteria at once.
The experiments use a simulated Universal Robots UR5e arm with a Robotiq gripper, pushing a T-shaped block on a tabletop in NVIDIA Isaac Lab [12]. Isaac Lab is able to run many environment instances in parallel on a single Graphical Processing Unit (GPU), which lets every model train at a fixed compute budget on equal terms. The policy does not command joint torques directly; instead, it selects an object-relative push primitive with four parameters: an approach offset, an approach angle, a push length, and a push direction. A cuRobo Inverse Kinematics (IK) solver then executes the chosen push. Goals lie in the planar rigid-body configuration space SE(2), and a push succeeds only when the object reaches a small position and orientation tolerance of the goal. Chapter 3 gives the full state, action, and reward definitions. The thesis poses three research questions, together with one supporting question on reward efficiency.

- **Research Question 1:** Does asymmetric self-play match single-agent training in a contact-rich planar goal-reaching task under a single-GPU budget?
- **Research Question 2:** Which of the five dimensions in the current setup accounts for the performance of the self-play agent?
- **Research Question 3:** What mechanism prevents self-play from generalising its training-time success?
- **Supporting Research Question:** Does potential-based reward shaping reach single-agent success with fewer parallel environments than the improvement reward?

The first question asks whether the method transfers to the contact-rich planar goal-reaching task at all, comparing self-play against the single-agent baseline under one budget and one success criterion. The second question decomposes the difference between the contact-rich planar goal-reaching task and the settings where self-play succeeded into five candidate dimensions: goal coupling, reward formulation, imitation signal, goal representation, and compute scale. Each is tested in turn. The comparison between a rotationally symmetric disc and the T-block isolates the coupled goal as a single variable. The third question examines the mechanism, using the within-distribution evaluation and the imitation signal. The supporting question isolates the contribution of reward shaping alone.

1.1 Contributions

This thesis makes three contributions. The first is the mechanism behind the self-play result. A within-distribution evaluation shows that the self-play policy solves 61.4 % of the disc goals and 11.7 % of the T-block goals the goal-proposing agent itself produces, against 7.3 % and 0.0 % of held-out scenes. The failure therefore separates into two compounding effects: the coupled multi-objective gate bounds competence on the proposer's own goals, and the narrow goal distribution prevents that competence from transferring. Five measurements support this: the

disc-against-T-block contrast, the within-distribution evaluation, the scale-sweep null result, the imitation clip-saturation measurement, and the goal-encoder ablation. The second contribution is the setup contrast. It sets this task against the settings where self-play succeeded, along five dimensions: goal coupling, reward formulation, imitation signal, goal representation, and compute scale. A negative result here is thereby explained as a difference in setup, not a denial of prior work done into self-play. The third contribution is the established task solvability. Potential-based reward shaping produces a single-agent policy that solves 79.3 % of held-out scenes with confidence intervals across five seeds, which anchors the self-play result as a property of the method rather than the task. The practical message follows: principled reward design should come before changes to the training dynamics. We scope every self-play result to this setup. Asymmetric self-play succeeds in the settings prior work used; the contribution here is the analysis of why it does not generalise in this one.

1.2 Thesis Outline

The remainder of this thesis is organised as follows. Chapter 2 reviews the background and related work, covering RL and PPO, potential-based reward shaping, curriculum learning and self-play, goal-conditioned learning, and the mechanics of pushing. That chapter also draws the mechanical contrast between the disc and the T-block, and explains the multi-objective nature of the goal. Chapter 3 defines the task and its Markov Decision Process, explains why a naive improvement reward underperforms, and derives the potential-based reward and its SE(2) form. It then describes the nine models grouped by role, and sets out the experiment set that produces the results. Chapter 4 presents the experimental setup: the simulation and training configuration, the phases of the experiment set, the held-out validation protocol, the cross-environment comparison, and an off-policy baseline. Chapter 5 reports the results for each research question, namely whether self-play succeeds here, which setup dimension accounts for the outcome, the underlying mechanism, and the environment-efficiency of shaping. Chapter 6 discusses the findings, states the limitations, and outlines future work. The appendices collect training curves, parameter sensitivity, scene descriptions, and detailed diagnostics.

2 Background and Related Work

This chapter develops the theory the thesis relies on and situates the work within the literature. It begins with the formal definitions of reinforcement learning, first for fully observable problems and then for the partially observable case that motivates the recurrent policy used here. It then describes the two algorithms used in the experiments, Proximal Policy Optimization and Soft Actor-Critic, before describing the two techniques the thesis compares: potential-based reward shaping, and structured training through curriculum learning and asymmetric self-play. The chapter ends with the mechanics and geometry of planar pushing, which explain why the task is multi-objective and why its two objectives cannot be optimised in isolation.

2.1 Reinforcement Learning

Reinforcement Learning (RL) is the branch of machine learning concerned with how an agent should act in an environment so as to maximise a cumulative scalar reward. Unlike supervised learning, the agent is never told the correct action; it receives only an evaluative signal and must discover, through trial and error, which behaviours lead to high long-term return. The agent observes the state of the environment, selects an action, and receives a reward together with a new state; repeating this loop produces a trajectory whose total reward the agent learns to improve. This interaction is formalised through the Markov Decision Process.

2.1.1 Markov Decision Processes

A Markov Decision Process (MDP) is the standard model of a fully observable sequential decision problem. Following [6, 13], it is a tuple $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$, where:

- $\mathcal{S}$ is the **state space**, the set of all configurations the environment can occupy. At time step t the agent perceives a state $s_t \in \mathcal{S}$.
- $\mathcal{A}$ is the **action space**, the set of actions available to the agent; in robotics this may be a set of joint commands or, as in this thesis, a set of parameterised motion primitives.
- $P(s_{t+1} \mid s_t, a_t)$ is the **transition function**, giving the probability of reaching s_{t+1} after taking action a_t in state s_t. It encodes the environment dynamics.
- $R(s_t, a_t, s_{t+1})$ is the **reward function**, the scalar feedback for a transition.
- $\gamma \in [0, 1]$ is the **discount factor**, which sets the relative weight of immediate against future reward.

The defining property is the *Markov* assumption: the transition and reward depend only on the current state and action, not on the history that preceded them. Formally, $P(s_{t+1} \mid s_t, a_t) = P(s_{t+1} \mid s_0, a_0, \ldots, s_t, a_t)$. The state is therefore a sufficient statistic of the past, which is what makes value functions and the Bellman recursion well defined.
The agent acts according to a **policy** $\pi(a \mid s)$, a mapping from states to a distribution over actions. Its objective is to maximise the **expected return**, the expected discounted sum of future rewards:

$$J(\pi) = \mathbb{E}_{\pi}\left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t, s_{t+1})\right]. \tag{1}$$

The discount factor γ has a dual role. Mathematically it keeps the infinite sum bounded; behaviourally it interpolates between a myopic agent ($\gamma \to 0$), which optimises only the immediate reward, and a far-sighted agent ($\gamma \to 1$), which weighs distant rewards almost as heavily as immediate ones. In episodic tasks such as the pushing problem studied here, the horizon is finite and the undiscounted case $\gamma = 1$ is admissible provided every trajectory eventually terminates, a subtlety that becomes important for reward shaping in Section 2.4.

Value functions. To reason about long-term consequences, RL introduces two value functions. The **state-value function** $V^\pi(s)$ is the expected return obtained by starting in state s and following π thereafter,

$$V^\pi(s) = \mathbb{E}_\pi\left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t, s_{t+1}) \;\middle|\; s_0 = s\right], \tag{2}$$

while the **action-value function** $Q^\pi(s, a)$ conditions additionally on taking action a first,

$$Q^\pi(s, a) = \mathbb{E}_\pi\left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t, s_{t+1}) \;\middle|\; s_0 = s,\, a_0 = a\right]. \tag{3}$$

The two are linked by $V^\pi(s) = \mathbb{E}_{a\sim\pi(\cdot|s)}[Q^\pi(s, a)]$, and their difference defines the **advantage** $A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$, which measures how much better action a is than the policy's average behaviour in state s. The advantage is central to the policy-gradient methods used in this thesis.

Bellman equations. Value functions satisfy a recursive consistency condition. For the action-value function,

$$Q^\pi(s, a) = \mathbb{E}_{s'\sim P(\cdot|s,a)}[R(s, a, s') + \gamma\, V^\pi(s')], \tag{4}$$

which decomposes the value of a state-action pair into the immediate reward and the discounted value of the successor state. The optimal value functions $V^*(s) = \sup_\pi V^\pi(s)$ and $Q^*(s, a) = \sup_\pi Q^\pi(s, a)$ induce the optimal policy $\pi^*(s) = \arg\max_a Q^*(s, a)$. Whether solved by dynamic programming, value iteration, or sampled approximation, essentially all RL algorithms can be seen as ways of estimating or improving the quantities in Equation 4.

Exploration versus exploitation. Because the agent learns from its own experience, it faces a fundamental trade-off. *Exploitation* chooses the action currently believed to be best, which secures known reward; *exploration* tries alternatives to improve those beliefs, at the risk of short-term loss for long-term gain. An agent that only exploits may converge to a suboptimal behaviour, while one that only explores never applies what it has learned. Mechanisms such as ϵ-greedy action selection and entropy regularisation, the latter central to Soft Actor-Critic (Section 2.3), exist to manage this trade-off. In sparse-reward manipulation the trade-off is difficult, because informative reward is encountered so rarely that undirected exploration is unlikely to reach it, a difficulty that both reward shaping and automatic curricula attempt to address.

2.1.2 Partial Observability and the POMDP

The MDP assumes the agent perceives the full state. Real and simulated robots rarely enjoy this: sensors are noisy, occlusions hide part of the scene, and quantities such as contact forces or friction coefficients are simply not observable. Such problems are modelled as a Partially

Observable Markov Decision Process (POMDP), a tuple $(\mathcal{S}, \mathcal{A}, P, R, \Omega, O, \gamma)$ that augments the MDP with an **observation space** Ω and an **observation function** $O(o \mid s', a)$ giving the probability of observation o after a transition to s'. The agent no longer sees s_t; it sees only o_t, and a single observation is generally not a sufficient statistic of the state.
Because the observation breaks the Markov property, an optimal policy cannot in general be a function of the current observation alone. The classical solution is to act on a **belief state**, the posterior $b_t(s) = P(s_t = s \mid o_0, a_0, \ldots, o_t)$ over states given the entire interaction history, which is itself Markovian; maintaining the exact belief, however, is intractable for continuous state spaces. A practical alternative, and the one adopted in this thesis, is to have the policy summarise the history with a recurrent network. A Long Short-Term Memory (LSTM) network maintains a hidden state that is updated at each step, which provides the policy with a learned, compressed approximation of the belief. In the pushing task the current observation reports object and goal poses, but not the underlying contact state or the outcome of prior pushes. The problem is therefore formally a POMDP. Conditioning the policy on the sequence of pushes through an LSTM lets it combine information across the episode, rather than respond to a single frame. For this reason every policy compared in this thesis is recurrent.

2.1.3 Model-Free and Model-Based Reinforcement Learning

RL algorithms divide into two families according to whether they build an explicit model of the environment dynamics $P(s_{t+1} \mid s_t, a_t)$ [14]. **Model-free** methods learn a policy or value function directly from sampled interaction, without ever estimating the transition function. They subdivide further:

- **Value-based** methods, such as Deep Q-Networks [15], learn an action-value function and act greedily with respect to it, obeying the recursion $Q(s, a) = \mathbb{E}[R(s, a, s') + \gamma \max_{a'} Q(s', a')]$.
- **Policy-based** methods, such as PPO [5], parameterise the policy directly and ascend the gradient of the expected return in Equation 1.
- **Actor-critic** methods, such as Soft Actor-Critic [16], combine the two, maintaining an actor that represents the policy and a critic that estimates value to reduce the variance of the policy gradient.

Model-free methods make no assumptions about the dynamics and are robust to complex, contact-rich physics, which makes them the default choice in manipulation; their weakness is high sample complexity, since every unit of information must be extracted from interaction.
Model-based methods instead learn an approximate dynamics model $\hat{P}(s_{t+1} \mid s_t, a_t)$ and use it to plan or to generate synthetic experience, typically by minimising a prediction error such as

$$L_{\text{model}} = \mathbb{E}_{\mathcal{D}}\left[\left\| P(s_{t+1} \mid s_t, a_t) - \hat{P}(s_{t+1} \mid s_t, a_t) \right\|^2\right], \tag{5}$$

over a dataset $\mathcal{D}$ of observed transitions. They can be far more sample-efficient, but their performance is bounded by the fidelity of the learned model, and small errors compound over long rollouts. The choice between the two families is therefore a choice between robustness and sample efficiency. This thesis works entirely in the model-free, on-policy setup and asks a complementary question: rather than learning a dynamics model to save samples, can a principled reward do so instead, by making each sampled transition more informative? Section 2.4 takes up this question.

2.2 Proximal Policy Optimization

Proximal Policy Optimization (PPO) [5] is the on-policy, model-free algorithm used to train every policy in this thesis. It belongs to the policy-gradient family, which optimises a parameterised policy $\pi_\theta(a \mid s)$ by ascending an estimate of the gradient of the expected return. The basic policy-gradient estimator,

$$\nabla_\theta J(\pi_\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\Big[\nabla_\theta \log \pi_\theta(a_t \mid s_t)\, \hat{A}_t\Big], \tag{6}$$

scales the score function of each action by its estimated advantage $\hat{A}_t$, so that actions better than average are made more likely. In practice the advantage is computed with Generalized Advantage Estimation (GAE), which trades bias against variance by exponentially weighting multi-step temporal-difference errors; GAE reappears in Chapter 3, where its decomposition explains how reward shaping interacts with a stale value estimate.

PPO improves on the earlier Trust Region Policy Optimization [17], which constrained each update to a trust region through a costly second-order optimisation. PPO achieves a similar effect with a first-order **clipped surrogate objective**. Writing the probability ratio between the new and old policies as $r_t(\theta) = \pi_\theta(a_t \mid s_t)/\pi_{\theta_{\text{old}}}(a_t \mid s_t)$, the objective is

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t\Big[\min\Big(r_t(\theta)\,\hat{A}_t,\ \text{clip}\big(r_t(\theta), 1-\epsilon, 1+\epsilon\big)\,\hat{A}_t\Big)\Big]. \tag{7}$$

The clipping removes the incentive to move the policy ratio beyond $[1-\epsilon, 1+\epsilon]$, which keeps each update inside an implicit trust region and prevents the large, destabilising steps that occur in unconstrained policy gradients. This same clipping mechanism is later reused in the behavioural-cloning loss of asymmetric self-play (Section 2.6), where it has an unintended consequence that this thesis diagnoses. The full PPO objective adds a value-function loss $L^{\text{VF}}(\phi) = \mathbb{E}_t[(V_\phi(s_t) - \hat{R}_t)^2]$ and an entropy bonus $L^{\text{ENT}} = \mathbb{E}_t[\mathcal{H}(\pi_\theta(\cdot \mid s_t))]$ that encourages exploration:

$$L(\theta) = L^{\text{CLIP}}(\theta) - c_1 L^{\text{VF}}(\phi) + c_2 L^{\text{ENT}}, \tag{8}$$

with coefficients c_1 and c_2 balancing the three terms.

2.3 Soft Actor-Critic

Soft Actor-Critic (SAC) [16] is an off-policy actor-critic algorithm built on the maximum-entropy framework. Where standard RL maximises return alone, SAC augments the objective with the entropy of the policy,

$$J(\pi) = \sum_{t=0}^{T} \mathbb{E}_{(s_t,a_t)\sim\rho_\pi}\big[R(s_t,a_t) + \alpha\,\mathcal{H}\big(\pi(\cdot \mid s_t)\big)\big], \tag{9}$$

where $\mathcal{H}(\pi(\cdot \mid s_t)) = \mathbb{E}[-\log \pi(\cdot \mid s_t)]$ is the policy entropy and the temperature α sets the weight of exploration against reward. Maximising entropy encourages the policy to remain as stochastic as the reward permits. This improves exploration and robustness, and discourages premature convergence to a single mode, an advantage in sparse-reward or multimodal problems. Being off-policy, SAC reuses past experience from a replay buffer, and is typically far more sample-efficient than on-policy methods. SAC is relevant here as the basis of an off-policy comparison baseline. Combined with Hindsight Experience Replay [18], which relabels failed trajectories with the goals they happened to achieve, it provides a natural goal-conditioned learner against which the on-policy results can be calibrated, as described in Chapter 4.

2.4 Potential-Based Reward Shaping

Reward shaping supplies an agent with additional reward beyond the sparse task signal, to guide learning through the temporal credit-assignment problem [19]. Its danger is equally well known: an ill-chosen shaping reward can change the optimal policy, so that the agent learns to exploit the shaping rather than to solve the task. [6] give the canonical illustration, a bicycle agent rewarded for progress toward a goal that learns to ride in circles near the start, collecting the progress reward repeatedly without ever advancing.

Potential-Based Reward Shaping (PBRS) is the principled resolution of this danger. [6] prove that a shaping reward of the form

$$F(s, a, s') = \gamma\,\Phi(s') - \Phi(s), \tag{10}$$

defined as the discounted difference of a **potential function** $\Phi : \mathcal{S} \to \mathbb{R}$ over states, leaves the optimal policy of the MDP unchanged for *any* choice of Φ. The intuition is that the shaping term telescopes along a trajectory: summed over an episode it collapses to a boundary term that depends only on the first and last states, so it cannot create the reward-collecting cycles that non-potential shaping permits. Formally, adding Equation 10 to the task reward shifts every action-value by the same state-dependent constant, $Q'(s, a) = Q(s, a) - \Phi(s)$, and since the shift is independent of the action it does not alter the arg-max that defines the optimal policy. This decoupling is the property the thesis exploits: it permits a dense, informative reward to be designed purely to accelerate learning, with a guarantee that the task being solved is unchanged.

Subsequent work extended the framework in directions this thesis uses. [20] generalised the potential to depend on time as well as state, $F(s, t, s', t') = \gamma\Phi(s', t') - \Phi(s, t)$. This preserves policy invariance while letting the shaping adapt during learning, and so licences shaping that changes across a curriculum. [21] showed that an arbitrary reward function can be expressed as potential-based advice, widening the class of admissible potentials. Crucial for the present setting is the episodic case. [22] and [19] analysed PBRS in the finite-horizon setting and clarified when the undiscounted case $\gamma = 1$ is safe. Provided every trajectory terminates and terminal states are handled consistently, the shaping sum telescopes to $\Phi(s_T) - \Phi(s_0)$ and remains bounded. This result justifies the use of $\gamma_{\text{shaping}} = 1$ in the potentials constructed in Chapter 3. The theorem and its episodic refinement are the foundation of the reward design in this thesis. The specific potentials, which combine bounded exponential functions of position and orientation error, are introduced there rather than here, since they are a contribution of the work rather than prior art.

2.5 Curriculum Learning and Automatic Curricula

An orthogonal approach to sample inefficiency is to change not the reward but the order in which the agent encounters tasks. Curriculum learning organises training from easy to hard, so that competence acquired on simple problems supports progress on difficult ones [7]. The survey of [7] provides a taxonomy that distinguishes, among other dimensions, *forced* curricula, in which a designer fixes the sequence of tasks or objectives in advance, from *automatic* curricula, in which the sequence is derived from the learner's own progress. Both appear in this thesis: a forced position-then-rotation curriculum, and an automatic curriculum generated by self-play.

Several forced and semi-automatic schemes inform the design. Reverse curriculum generation [23] starts the agent near the goal and expands the start distribution outward as competence

increases, so that the agent always trains at the boundary of what it can already achieve. Precision-based curricula such as Continuous Curriculum Learning [24] progressively tighten the success tolerance, which is a close analogue of staging a pushing task from coarse positioning to fine orientation control. The survey of [25] organises automatic curriculum methods into families that include self-paced learning, teacher-student setups, and goal-generation, and notes the recurring difficulty that an automatic curriculum can propose goals faster than the learner masters them. That failure mode is the one the self-play experiments in this thesis encounter.

2.6 Asymmetric Self-Play

Asymmetric Self-Play (ASP) is the automatic curriculum at the centre of the thesis. Introduced by [8], it pits two policies embodied in the same agent against each other: Alice proposes a task by acting in the environment, and Bob must then solve it. Because Alice demonstrates each task by performing it, every proposed goal is achievable by construction, and the competition between the two produces a curriculum of increasing difficulty with no external reward or human specification. Sukhbaatar et al. define a self-regulating reward structure,

$$R_B = -\gamma\, t_B, \tag{11}$$

$$R_A = \gamma \max(0,\, t_B - t_A), \tag{12}$$

where t_A is the time Alice takes to set the task, t_B the time Bob takes to solve it, and γ a scaling coefficient. Alice is rewarded when Bob is slow but penalised for her own effort, so her optimal behaviour is to find the *simplest* tasks that Bob cannot yet solve, placing each new task just beyond his current ability. This time-based reward is one of the two Alice reward formulations the thesis evaluates.

[9] scaled ASP to robotic manipulation and introduced the second formulation. They model the problem as a goal-augmented MDP $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \mathcal{G} \rangle$ with an explicit goal space $\mathcal{G}$, run Alice to produce a trajectory whose final state s_T becomes the goal for Bob, and train Bob from the same initial state on the resulting goal distribution $\mathcal{G}(g \mid s_0)$. Their Alice reward is *outcome-based* rather than time-based: Alice receives a reward when Bob fails to reach the goal, a shaping bonus for proposing a valid goal, and a penalty for placing objects out of bounds, while Bob receives a sparse reward for reaching the goal. Both the outcome-based and the time-based Alice rewards are compared in this thesis.

A distinctive feature of the manipulation formulation is that Alice's own trajectory is itself a demonstration of the goal she sets, which [9] use through Alice Behavioral Cloning (ABC). Bob is trained on a combined objective $\mathcal{L} = \mathcal{L}_{\mathrm{RL}} + \beta \mathcal{L}_{\mathrm{abc}}$, where the behavioural-cloning term imitates Alice's relabelled trajectory on goals Bob failed to solve. To prevent the imitation term from producing destabilising policy updates, they apply the PPO clipping mechanism of Equation 7, with the advantage fixed to one:

$$\mathcal{L}_{\mathrm{abc}} = -\,\mathbb{E}_{(s_t, g_t, a_t) \in \mathcal{D}_{\mathrm{BC}}} \left[\mathrm{clip}\left(\frac{\pi_B(a_t \mid s_t, g_t; \theta)}{\pi_B(a_t \mid s_t, g_t; \theta_{\mathrm{old}})},\, 1-\epsilon,\, 1+\epsilon \right) \right]. \tag{13}$$

This clipped behavioural-cloning loss behaves as intended only when Alice's demonstrated actions lie close to Bob's current policy. When the two diverge strongly, as they do under the discrete, contact-rich action space of this thesis, the clip saturates and the imitation gradient vanishes; this interaction is analysed in the results. Behavioural cloning more broadly provides the wider context for ABC. This includes cloning from observation [26], implicit energy-based

cloning [27], and demonstration-boosted RL [28, 29], all methods for adding a demonstration signal to an RL objective.
The broader set of automatic curricula based on adversarial goal generation places ASP among several related methods. PAIRED [30] generates environments that maximise the regret between a protagonist and an antagonist. AMIGo [31] trains a teacher to propose goals a student finds challenging but achievable. Adversarial Intrinsic Motivation [32] directs exploration with a Wasserstein distance to the goal distribution. All share ASP's central premise, that a well-tuned adversary produces a useful curriculum, and all share its central risk, that the adversary may generate tasks the learner cannot use. The stability of such coupled optimisation is itself a subject of study. [33] decompose the dynamics of differentiable games into potential and Hamiltonian components, and identify when adversarial training converges or cycles. [34] demonstrate that competitive self-play can reach very high performance, but only at very large scale. These two observations define the scope of the present study: the self-play results here are obtained under a single-Graphical-Processing-Unit budget, and are interpreted accordingly.
The goal criterion also separates this thesis from the settings in which ASP succeeded, and the difference determines what the method must achieve. In the original formulation of [8], a goal is a single state Bob must reach, often in a domain where position dominates and orientation is absent or loosely constrained. In the manipulation setting of [9], a goal is the final pose of Alice's trajectory, scored within a tolerance on each component. The task studied here instead applies a strict combined gate. A goal counts as reached only when position and orientation both lie within tolerance at the same time. The contact mechanics of Section 2.8 couple the two, so neither can be optimised alone. The goal is therefore genuinely multi-objective in a sense the prior settings did not require. This is a difference in setup, not a defect in the earlier work, and it defines one of the dimensions along which this thesis explains why a proven method underperforms here.

2.7 Goal-Conditioned and Hierarchical Reinforcement Learning

ASP produces a goal-conditioned policy, and the way a goal is represented influences what the policy can learn. Goal-conditioned RL augments the state with a goal and learns a single policy $\pi(a \mid s, g)$ that generalises across goals; Hindsight Experience Replay [18] makes this practical under sparse reward by relabelling achieved states as goals. Sukhbaatar et al. in their goal-embedding work [35] showed that compressing goals into a learned latent through a goal encoder can improve hierarchical learning, an idea the thesis adopts in the form of the GoalEncoder used by Bob. The effect of such a compression can be positive or negative, and the information-bottleneck literature explains why. [36] formalise the trade-off between compression and predictive sufficiency. InfoBot [37] applies it in RL, and shows that a bottleneck can either improve generalisation by discarding distractors or reduce precision by discarding task-relevant detail. Whether the goal encoder improves or reduces performance in the pushing task is one of the questions the ablations address.
Hierarchical RL provides further context for the two-level Alice-Bob structure. Methods such as HIRO [38] and FeUdal Networks [39] decompose control into a high-level policy that sets sub-goals and a low-level policy that achieves them. The Option-Critic architecture [40] learns temporally extended actions with their own termination conditions, a useful lens on the push primitive used here as a macro-action. The survey of [41] situates these approaches and their open challenges. Imagined-goal methods [42] generate goals from a learned latent, mirroring how Alice's proposals define Bob's training distribution.

2.8 Mechanics and Geometry of Planar Pushing

The final body of theory explains why the pushing task is genuinely multi-objective. When a robot slides an object across a surface, the motion is governed by friction between the object and the support. The analysis is made tractable by the **quasi-static** assumption: pushing is slow enough that inertial forces are negligible and the applied load is balanced entirely by friction [10]. Under this assumption the object moves only while it is being pushed and stops the instant the push stops, so its trajectory is determined by the geometry of frictional contact rather than by dynamics.

The central tool is the **limit surface** [11]. At a sliding contact, the set of all friction forces and moments the interface can sustain forms a convex surface in the three-dimensional load space of planar forces and moment, $P = [F_x, F_y, M]$. For isotropic Coulomb friction this surface is a convex, closed shape often approximated by an ellipsoid. Its importance lies in the **normality (maximum-power) principle**: the object's instantaneous motion twist $\mathbf{t} = [v_x, v_y, \omega]$, comprising a linear velocity and an angular velocity, is normal to the limit surface at the point corresponding to the applied load,

$$\mathbf{t} \;\propto\; \nabla f(\mathbf{w}), \tag{14}$$

where $f(\mathbf{w}) = 0$ is the limit surface and $\mathbf{w}$ the applied wrench. The consequence is important for this thesis. Consider any push not directed exactly through the object's centre of friction. Its wrench lies where the surface normal has a non-zero moment component, so the resulting twist contains an angular velocity: *almost every push rotates the object as well as translating it.* Translation and rotation are mechanically coupled, and a goal that constrains both cannot be decomposed into two independent sub-problems. This coupling is the physical origin of the combined position-and-orientation success criterion that recurs throughout the results. It is also the reason a reward or curriculum that treats the two objectives as separable is inconsistent with the underlying physics. [43] developed the controllability and planning consequences of this mechanics for stable pushing, and [44] gave practical force-motion models. The survey of [45] and the high-fidelity dataset of [46] document how strongly real pushing behaviour depends on object shape and pressure distribution.

This dependence on object shape distinguishes the two objects used in this thesis and motivates one of its experiments. A rotationally symmetric disc has a radially symmetric pressure distribution about its centre. A push whose line of action passes through that centre applies a wrench with no moment component, so the limit-surface normal points opposite the push direction and the resulting twist is a pure translation. The disc therefore has no orientation to control, and a goal on the disc constrains position alone. The T-block has an asymmetric mass distribution, and few pushes pass through its centre of friction; each off-centre push carries a moment, and by the normality condition of Equation 14 the resulting twist contains an angular velocity. Orientation is then an independent quantity that must be controlled alongside position. This mechanical difference is the basis of the coupling-isolation experiment of Chapter 3. The disc removes the orientation objective at the level of the physics. A self-play model that reaches the single-agent level on the disc, yet underperforms it on the T-block, thereby identifies the coupled objective, rather than the object itself, as the source of the difficulty.

Because position and orientation are coupled, it is natural to measure goal error with a single metric on the space of planar poses, rather than with two separate quantities. The configuration of a planar rigid body is an element of the **Special Euclidean group** SE(2), the group of rigid-body transformations of the plane. Its elements combine a translation $(x, y) \in \mathbb{R}^2$ with a

rotation $\theta \in \mathrm{SO}(2)$. [47] formulated rigid-body kinematics and dynamics in the language of Lie groups, which provides the tools to define a distance on this space. A left-invariant Riemannian metric on SE(2) yields a geodesic distance between two poses. Because translation is measured in metres and rotation in radians, combining them requires a **characteristic length** L that converts an angular error into an equivalent linear displacement. A pose error can then be summarised by a single scalar of the form

$$d_{\text{pose}} \;=\; \sqrt{\Delta x^2 + \Delta y^2 + L^2\,\Delta\theta^2}\,, \tag{15}$$

in which L sets the relative weight of orientation against position and is naturally identified with a physical length scale of the object. This unified SE(2) distance underlies the single-potential reward used for the self-play variants in Chapter 3. The idea of respecting the geometry and symmetry of the configuration space also appears in recent manipulation learning: SE(3)-equivariant and diffusion-based methods [48], MDP homomorphic networks [49], equivariant transporter networks [50], and equivariant RL under partial observability [51] all exploit the group structure of the task, and provide broader context for a geometry-aware treatment of planar pushing.

2.9 Positioning of This Work

The literature provides two well-founded but rarely compared techniques for reducing the sample cost of on-policy manipulation learning: shaping the reward, with the policy-invariance guarantee of PBRS, and structuring the training process, through forced or automatic curricula such as ASP. Each has strong theoretical support and prominent empirical results, yet the two are seldom evaluated directly against each other on a single task, under a common budget, and against a common success criterion. The planar-pushing setting makes the comparison more stringent, because the limit-surface coupling of Equation 14 makes the task irreducibly multi-objective and therefore places a strong demand on any method that implicitly assumes the two objectives can be handled separately. The chapters that follow implement both techniques in this setting and measure what each contributes.

3 Methods

This chapter defines the goal-reaching task and the nine models that are compared on it. It begins with the simulation environment and the formal Markov Decision Process, including the object-relative push primitive that serves as the action. It then explains why the hand-tuned improvement reward of an initial baseline underperforms, and derives the potential-based reward that replaces it. The reward is presented first as separate position and orientation potentials, then as a single Special Euclidean group SE(2) distance. The chapter closes with the models, grouped by the role each one serves, and with an analysis of why potential-based shaping partially assists self-play without matching single-agent training.

3.1 Simulation Environment

All experiments are performed in NVIDIA Isaac Lab 2.3.0, built on Isaac Sim 5.1.0, using the PhysX physics engine on the Graphical Processing Unit (GPU) [12, 52]. The agent is a Universal Robots UR5e arm fitted with a Robotiq 2F-140 parallel-jaw gripper. The manipulated object is a T-shaped block resting on a tabletop, and the goal is displayed as a coloured ghost of the same block at the target pose. Figure 1 shows the rendered environment, and Figure 2 the setup schematically.

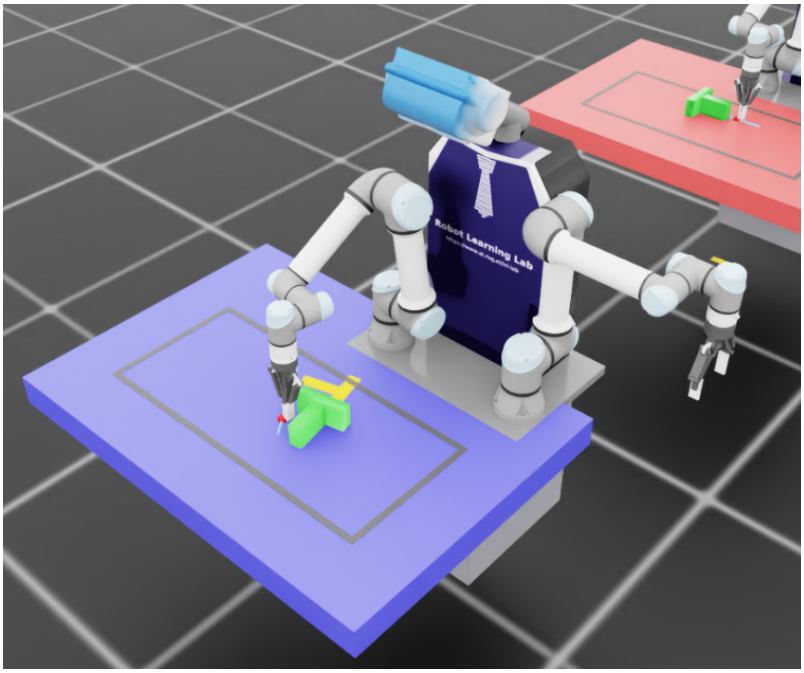

Figure 1: The simulated environment in Isaac Lab: the UR5e arm, the tabletop workspace, the T-block, and the goal ghost.

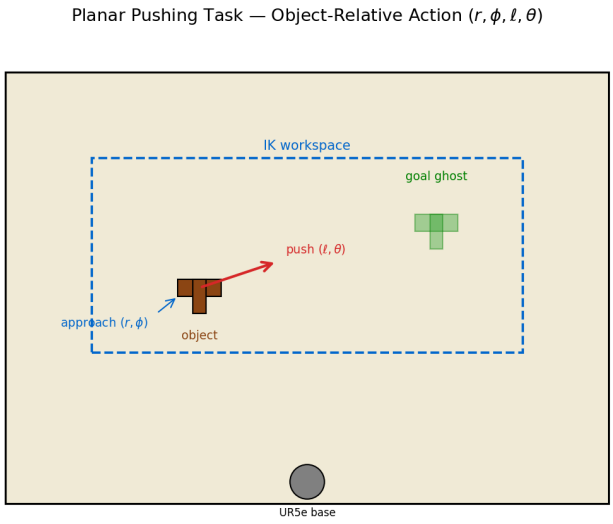

Figure 2: The planar-pushing task. A UR5e arm pushes a T-block (blue) across a tabletop toward a goal pose (orange). Goals are defined in SE(2) and combine a target position and a target orientation.

The principal advantage of Isaac Lab for this study is GPU-batched parallelism. Many copies of the environment are stepped at the same time on a single device, which produces the large sample batches on-policy learning requires. Figure 3 shows a render of the batched simulation. The single-agent and curriculum models are trained with 528 parallel environments, which yields a batch of roughly 7,920 push transitions per Proximal Policy Optimization (PPO) update. A hardware upper bound of one RTX Pro 6000 GPU (96 GB of memory) sets the compute budget for every model, so all methods are compared on equal terms. Inverse Kinematics (IK) for the arm is solved with cuRobo 0.7.5, a GPU-accelerated batched IK solver.

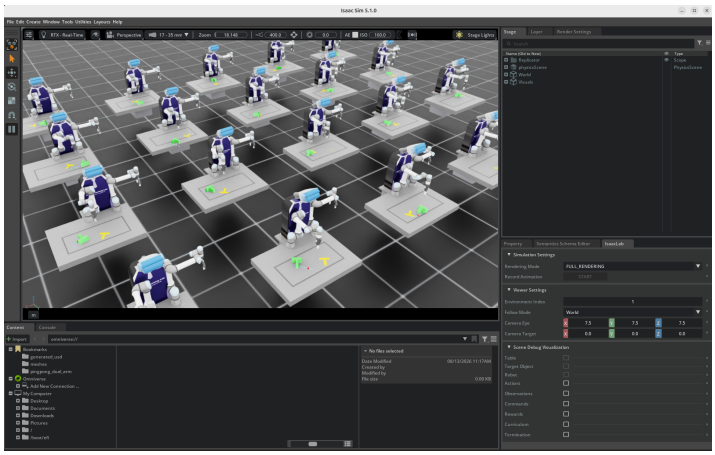

Figure 3: A render of the batched simulation, showing 20 of the parallel environments; the training runs used between 256 and 2,048.

3.2 The Planar-Pushing Markov Decision Process

The task is modelled as an episodic Markov Decision Process (MDP) with a macro-action, so that a single decision corresponds to one complete push rather than to a low-level joint

command.

State. The state is a 28-dimensional vector composed of four groups. These are the end-effector pose (6 values), the object pose and velocity (14 values), the goal pose (6 values), and the two scalar position and orientation errors between object and goal. As explained in Section 2.1.2, the current observation does not report the underlying contact state. The problem is therefore formally partially observable, and every policy is recurrent.

Action. The policy does not command joint torques. It selects an *object-relative push primitive* described by four parameters: a radial approach offset r, an approach angle ϕ in the object frame, a push length ℓ, and a push direction θ in the world frame. Each parameter is discretised into 21 bins, so the action is a four-dimensional MultiCategorical variable with $21^4 = 194{,}481$ discrete values. Section 3.3 describes how the primitive is decoded into a robot motion and executed.

Transition. The object moves according to the quasi-static pushing mechanics of Section 2.8. The limit-surface normality condition of Equation 14 couples translation and rotation: almost every push changes both the position and the orientation of the block. This coupling is a property of the environment, not a modelling choice. It is the reason the task cannot be decomposed into independent position and orientation sub-problems.

Reward. The reward is the subject of Sections 3.4 to 3.6. It is the single component that differs across the models compared in this thesis, which is what makes the comparison a controlled one.

Success criterion. A push episode is counted as successful when the object is within 0.05 m of the goal position *and* within 0.2 rad of the goal orientation. This combined gate is applied identically in training and evaluation. The two conditions must hold at the same time, and the physics couples the two axes. The combined gate is therefore the central difficulty of the task, and the quantity every model is ultimately measured against.

3.3 The Object-Relative Push Primitive

Each decision commands one complete push rather than a joint torque, so the learning problem is defined over whole pushing motions. The four parameters are read in the object frame wherever possible, which is the property that makes the primitive robust to the object's location. The radial approach offset r and the approach angle ϕ place the contact point on the object's boundary. The angle ϕ selects a direction around the object, and r sets how far outside the boundary the arm first descends. The push length ℓ and the push direction θ then define the straight motion that follows contact. Because the contact point is expressed relative to the object, the arm meets the object wherever it lies on the table, and the contact rate stays near 95 %.

This choice is deliberate, and it addresses two shortcomings of an absolute parameterisation in table coordinates. The first concerns exploration. When the push start was specified in world coordinates, a randomly sampled action contacted the object on only about 2 % of pushes. The remaining pushes missed, returned no progress signal, and left the policy with almost nothing to learn from; in self-play the goal-proposing agent could not begin to improve. Placing the contact point on the object boundary instead makes contact the default, so nearly every push becomes an informative transition. The second concerns generalisation. With a fixed object position and absolute coordinates, the policy could memorise a single approach point and

ignore the observation. Tying the action to the current object pose removes that shortcut, and requires the policy to read where the object is, so it acts on objects placed anywhere on the table. The push direction θ is kept in the world frame, aligned with the relative goal-direction feature of the observation, so the direction that reduces the goal distance is straightforward to learn. Each parameter is discretised into 21 bins, and the bins map linearly onto physical ranges bounded by the reachable workspace. Once decoded, the primitive runs as a five-phase Cartesian trajectory. The arm approaches a point above the contact location, descends to the table, pushes along θ for a distance ℓ, retracts, and returns to a neutral pose. The whole motion spans 72 physics substeps with the gripper held closed. Each Cartesian waypoint is mapped to arm joint targets by the cuRobo inverse-kinematics solver. A waypoint that falls outside the region where the solver returns a valid arm configuration is clamped to the boundary, so the credit assigned to an action matches the motion actually performed. Figure 4 shows the discrete choices the primitive exposes to the policy: the object-relative grid of contact points, and the world-frame push directions and lengths. Algorithm 1 summarises the decoding and execution; the workspace bounds and the five phases are detailed in Appendix E.

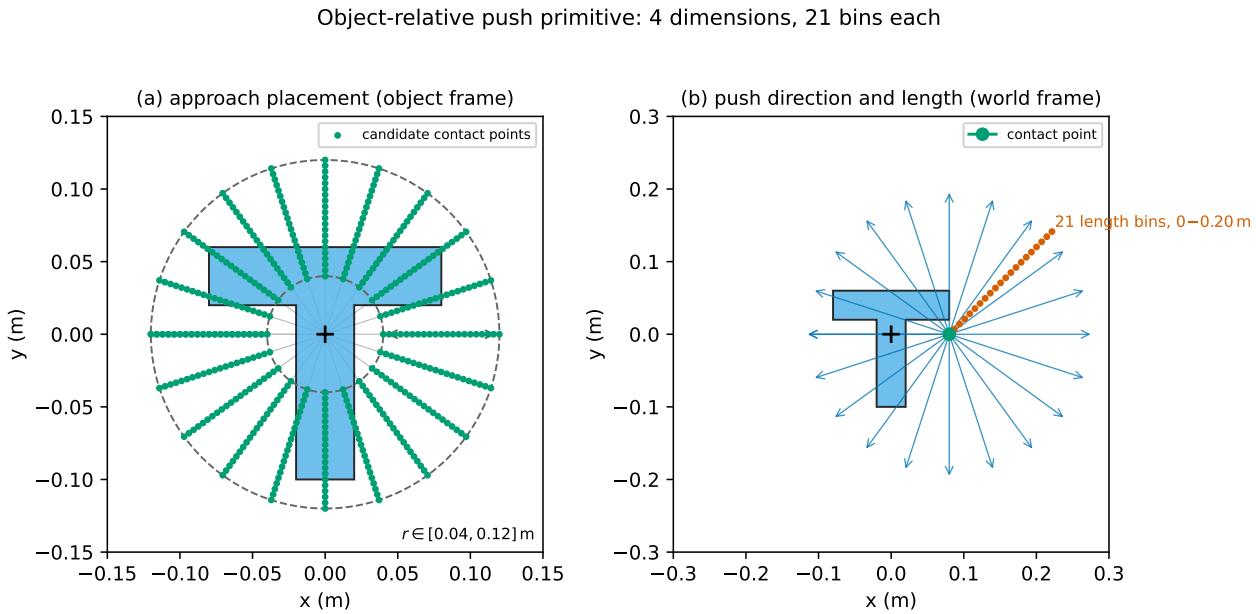

Figure 4: The push primitive drawn to scale over a top-down T-block. Panel (a): the object-relative grid of candidate contact points (green), bounded by a radial band (dashed circles). Panel (b): the world-frame push, one of 21 headings (blue arrows) with lengths up to 0.20 m (marked along one heading).

Algorithm 1 Decode and execute one object-relative push primitive.

Require: action bins $(b_r, b_\phi, b_\ell, b_\theta) \in \{0, \dots, 20\}^4$; object pose (x_o, y_o, ψ_o)

map each bin to its physical value: offset r, angle ϕ, length ℓ, direction θ
$p_{\text{contact}} \leftarrow$ boundary point of the object at angle ϕ, backed off by r
$p_{\text{start}} \leftarrow$ world position of p_{contact}
$p_{\text{end}} \leftarrow p_{\text{start}} + \ell\,(\cos\theta, \sin\theta)$
clamp p_{start} and p_{end} to the reachable workspace
for phase in {approach, descend, push, retract, return} **do**
    compute the Cartesian waypoint for the phase
    solve arm joint targets for the waypoint with cuRobo IK
    step the simulator with the gripper held closed
end for
return the object pose after the push

3.4 Why the Improvement Reward Underperforms

An initial baseline used a hand-tuned reward based on fractional improvement. It rewards each push in proportion to the fraction of the remaining error it removes. Let d_{prev} and d_{now} be the position error before and after a push, and y_{prev}, y_{now} the orientation error. The reward is then

$$R = \alpha \, \frac{d_{\text{prev}} - d_{\text{now}}}{d_{\text{prev}}} - \beta_{\text{pos}} \, d_{\text{now}} - \beta_{\text{rot}} \, y_{\text{now}}, \qquad \alpha = 3.0,\ \beta_{\text{pos}} = 0.5,\ \beta_{\text{rot}} = 0.25. \tag{16}$$

This reward performs poorly for two related reasons. Both concern the gradient it supplies to PPO, not the optimal policy it defines. Consider a representative push that moves the object 1 cm closer to a goal 30 cm away. The improvement term contributes $3.0 \times 0.01/0.30 \approx 0.10$. The two penalty terms contribute $-0.5 \times 0.29 \approx -0.145$ and $-0.25 \times 0.5 = -0.125$, for a net reward of about -0.17. A push that makes genuine progress therefore receives negative reward, because the standing penalty on the remaining distance dominates the small improvement. This is the first problem: the reward is negative for most useful pushes, so the policy gradient points away from goal-directed behaviour.

The second problem is near-zero variance. Across a rollout, roughly 99 % of pushes produce a reward in the narrow interval $[-0.5, +0.1]$, and the sparse completion bonuses fire on about 1 % of pushes. At 528 environments and 15 pushes per rollout, only about 79 bonus events occur per iteration, too few to overcome the noise in the dense term. The Generalized Advantage Estimation (GAE) advantage is then dominated by noise, and the gradient is uninformative. The failure is therefore one of gradient quality and reward variance, not of a mis-specified optimum. The reward defines a reasonable objective, yet it supplies a poor learning signal.

3.5 Potential-Based Reward Shaping for Pushing

The remedy is to replace the improvement reward with a potential-based reward. The per-push signal is then dense, correctly signed, and bounded, while the optimal policy is provably unchanged (Section 2.4). The design uses two bounded exponential potentials, one for position

and one for orientation:

$$\Phi_{\text{pos}}(s) = w_{\text{pos}} \exp\left(-k_p\, d^2\right), \tag{17}$$

$$\Phi_{\text{rot}}(s) = w_{\text{rot}} \exp(-k_r\, c(s)), \qquad c(s) = \frac{1-\cos(\theta^* - \theta)}{2}, \tag{18}$$

where d is the planar position error and $c(s)$ is a cosine orientation distance. The cosine form is smooth and free of the discontinuity a modular angle difference would introduce at $\pm\pi$; θ^* and θ are the goal and object yaw. The parameters are $k_p = 30$, $k_r = 5$, and $w_{\text{pos}} = w_{\text{rot}} = 10$. The dense reward on a transition is the difference of the summed potential,

$$F(s, s') = \left[\Phi_{\text{pos}}(s') - \Phi_{\text{pos}}(s)\right] + \left[\Phi_{\text{rot}}(s') - \Phi_{\text{rot}}(s)\right], \tag{19}$$

which is Equation 10 with $\gamma_{\text{shaping}} = 1$. The undiscounted setting is admissible because the task is episodic and every episode terminates, as established for episodic MDPs by [19]. Two sparse bonuses are layered on the dense term. A completion bonus of $+5$ is given when the position gate is satisfied, and a further $+2$ when the combined gate is satisfied. A penalty of -5 is applied if the block tips over. These bonuses are not themselves potential-based, but they fire only at terminal or near-terminal events, where they do not create the reward-collecting cycles that Section 2.4 identifies as the danger of non-potential shaping.
On the same representative push, the potential-based reward inverts the sign of the baseline. For a 1 cm improvement from 30 cm, the position potential difference is $10 \times [\exp(-30 \times 0.29^2) - \exp(-30 \times 0.30^2)] \approx +0.13$. Every push toward the goal now receives positive reward, and every push away from it negative reward. Because the potential is bounded in $[0, w]$, the reward magnitude cannot be inflated by a physics contact spike. Figure 5 contrasts the per-push reward of the two schemes over a representative episode, and shows the potential landscape and the gradient comparison.

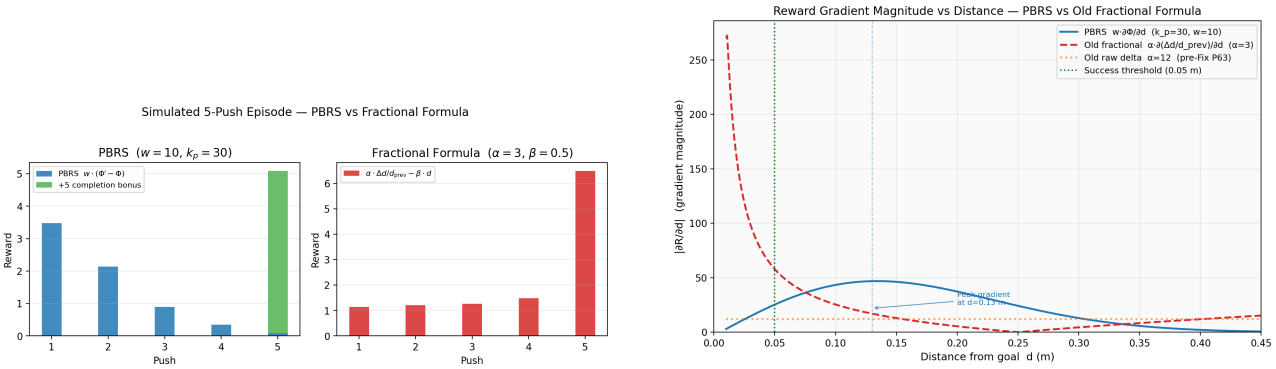

Figure 5: Left: per-push reward over a representative five-push episode for the improvement reward and the potential-based reward. The improvement reward is negative or near-zero for most pushes; the potential-based reward is correctly signed on every push. Right: the gradient supplied by the two schemes as a function of distance to the goal.

The reward is implemented as a potential-based reward function. Four properties make it suitable for a contact-rich task, and all follow from its form. It is Markovian, since it depends only on the current and successor state. It is bounded. It produces zero net reward over any cycle that returns to the same state, which removes the incentive for reward-collecting loops. And it produces zero reward while the object is stationary, so it imposes no penalty for deliberation.

3.6 The SE(2) Unified Distance

The two-potential reward treats position and orientation as separate terms whose gradients can point in different directions. For the self-play models (Section 3.7) the reward instead uses a single SE(2) distance, so one potential governs the whole pose. Following the Lie-group formulation of Section 2.8, the pose error is summarised by the scalar

$$d_{\text{pose}} = \sqrt{\Delta x^2 + \Delta y^2 + L^2\,\Delta\theta^2}, \tag{20}$$

where Δx, Δy are the position error components and $\Delta\theta$ the orientation error. Here L is a characteristic length that converts an angular error into an equivalent linear displacement. For the T-block $L = 0.07\,\text{m}$, the block's radius of gyration; it is a physical property of the object, not a tuning parameter. The corresponding single potential and its dense reward are

$$\Phi_{\text{pose}}(s) = w\,\exp\!\left(-k\,d_{\text{pose}}^2\right), \qquad F(s, s') = \Phi_{\text{pose}}(s') - \Phi_{\text{pose}}(s), \tag{21}$$

with $k = 30$ and $w = 10$. The success threshold for this metric is $d_{\text{pose}} < 0.055\,\text{m}$ for the T-block. The single potential removes the competing-gradient problem of the two-potential form. A push that reduces d_{pose} is rewarded whether the reduction comes from position or orientation, so the reward is aligned with the coupled physics of the task. Both the two-potential and the unified formulations share the same reward implementation.

3.7 The Models

All nine models share the same backbone: a single PPO agent with an LSTM of hidden size 256 followed by a multilayer perceptron, and a MultiCategorical head over the four-dimensional, 21-bin push primitive. They differ only in the reward and the training dynamics, which is what makes the comparison controlled. The models fall into four groups by the role each serves. The single-agent baselines establish that the goal-reaching task is solvable and measure the reward-efficiency result. The self-play subjects are the primary objects of study: the two variants that underperform the baselines on the coupled goal. The coupling-isolation models repeat self-play on a rotationally symmetric disc, whose position-only goal removes the coupled success criterion. The reward-structure control and the cross-environment counterparts test two further explanations for the self-play result. Table 1 lists all nine models and what each is meant to show.

Model	Group	Reward and training dynamics	Envs	What it is meant to show
PPO-Baseline	Single-agent baseline	Improvement reward (Equation 16); one agent	256–2,048	The goal is reachable without shaping; sets the reference the potential-based reward is measured against.
PPO-PBRS	Single-agent baseline	Two-potential PBRS over position and orientation; one agent	256–2,048	The solvable baseline; reaches success with far fewer environments than the improvement reward.
PPO-Curriculum	Single-agent baseline	PBRS with forced position-then-orientation staging	528	Whether staging the two objectives helps once the reward is well shaped.
ASP-dPose	Self-play subject	SE(2) PBRS for Bob; outcome-based Alice reward; imitation on	256, 528, 2,048	Primary subject: outcome-reward self-play on the coupled T-block goal; trained up to 2,048 environments to test compute scale.
TASP-dPose	Self-play subject	SE(2) PBRS for Bob; time-based Alice reward; imitation on	256, 528, 2,048	Primary subject; isolates Alice's reward against ASP-dPose, and is likewise scaled to 2,048 environments.
ASP-disc	Coupling isolation	Outcome-based self-play on a disc; position-only goal	528	Whether self-play reaches the baseline level once orientation is unconstrained.
TASP-disc	Coupling isolation	Time-based self-play on a disc; position-only goal	528	Time-reward counterpart of the disc isolation.
Bob-penalty	Reward-structure control	Symmetric self-play reward; Bob penalised for its own solve time	528	Whether a symmetric reward structure collapses the self-play loop.
gym counterparts	Cross-environment	Single agent, curriculum, and self-play in the two-dimensional simulator	64, 256	Whether the ordering of the methods holds in a second simulator.

Table 1: The nine models share the same PPO, LSTM backbone, and object-relative push primitive; only the reward and training dynamics differ. TASP-dPose and Bob-penalty report four held-out seeds each; the missing validation is noted in Section 4.2. The full per-phase counts appear in Section 3.9 and Chapter 4.

PPO-Baseline. A single PPO agent trained with the improvement reward of Equation 16. It serves as the reference point for the reward comparison of Section 3.4 and is trained at 2,048 parallel environments, a larger batch than the other models use.

PPO-PBRS. A single PPO agent trained with the two-potential reward of Equations 17 to 19. This is the simplest model that uses the potential-based reward, with no curriculum and no second agent, and it is trained at 528 environments.

Algorithm 2 states the single-agent training loop shared by PPO-Baseline and PPO-PBRS; the two differ only in the reward R.

Algorithm 2 Single-agent training (PPO-Baseline and PPO-PBRS).

Require: reward R; environments N; iterations T; pushes per rollout H; epochs K
initialise recurrent policy π_θ and value function V_φ
for iteration $= 1$ to T **do**
  for each of the N environments in parallel **do**
    sample an object start pose and a goal pose
    for $t = 1$ to H **do**
      observe state s_t; sample push $a_t \sim \pi_\theta(\cdot \mid s_t)$
      execute a_t (Algorithm 1); observe s_{t+1}
      $r_t \leftarrow R(s_t, s_{t+1})$
    end for
  end for
  estimate advantages with Generalized Advantage Estimation
  for epoch $= 1$ to K **do**
    update π_θ and V_φ with the clipped PPO objective
  end for
end for
return π_θ

Within a rollout, each environment plays one episode of up to five pushes, or fewer when the combined gate is reached early; the environment then resets to a fresh start pose and goal pose and plays the next episode. A rollout of $H = 15$ pushes therefore holds up to three episodes.

PPO-Curriculum. PPO-PBRS with a forced three-phase curriculum that stages the two objectives. Phase 1 trains position only, with the orientation weight set to zero. Phase 2 begins when the episodic position-only success rate reaches 0.5 over a 20-iteration window; the orientation weight is then increased from 0 to 10 over 200 iterations. Phase 3 is the full two-objective reward. The trigger gates on the episodic position-only success rate, rather than on a per-push error mean. This is the corrected version of a trigger that in an earlier implementation never activated; the earlier failure and its correction are documented in Appendix **??**. Algorithm 3 shows the staging around the single-agent loop.

Algorithm 3 Forced curriculum staging (PPO-Curriculum).

Require: potential-based reward with orientation weight w_{rot}; iterations T

$w_{\text{rot}} \leftarrow 0$; phase $\leftarrow 1$ ▷ Phase 1: position only
for iteration $= 1$ to T **do**
  run one iteration of Algorithm 2 with the current w_{rot}
  if phase $= 1$ and the position-only success rate ≥ 0.5 over the last 20 iterations **then**
    phase $\leftarrow 2$ ▷ Phase 2: raise the orientation weight
  end if
  if phase $= 2$ **then**
    $w_{\text{rot}} \leftarrow \min(10,\ w_{\text{rot}} + 10/200)$
    if $w_{\text{rot}} = 10$ **then**
      phase $\leftarrow 3$ ▷ Phase 3: full two-objective reward
    end if
  end if
end for
return π_θ

ASP-dPose. An asymmetric self-play model that uses the SE(2) reward of Equation 21 for the goal-solving agent, Bob. The loop is shown in Figure 6, with its rendered counterpart in Figure 7. The goal-setting agent, Alice, performs five pushes to propose a goal. The goal is accepted only if it passes a validity check: the block has moved sufficiently, remains on the table, and lies inside the workspace. Bob then attempts the goal from a clean reset over up to ten pushes. Alice's reward is *outcome-based*, following [9]: she receives a reward when Bob fails, a shaping bonus for a valid goal, and a penalty for an out-of-bounds goal. Bob additionally uses a GoalEncoder, a small multilayer perceptron that maps the six-dimensional goal pose into an eight-dimensional latent, in the difference variant of [35]. Bob is trained with Alice Behavioral Cloning at $\beta = 0.5$. This imitation term is enabled in the canonical model; a variant with it disabled is retained only as an ablation.

TASP-dPose. Identical to ASP-dPose except that Alice's reward is *time-based*, following [8]: $R_A = \gamma_{sp} \max(0, t_B - t_A)$. This rewards Alice for proposing goals that take Bob more pushes to solve than Alice took to set, subject to a penalty on Alice's own effort. It is the self-regulating reward of Equation 12. The two self-play models therefore differ only in Alice's reward, which isolates the effect of the goal proposal mechanism.

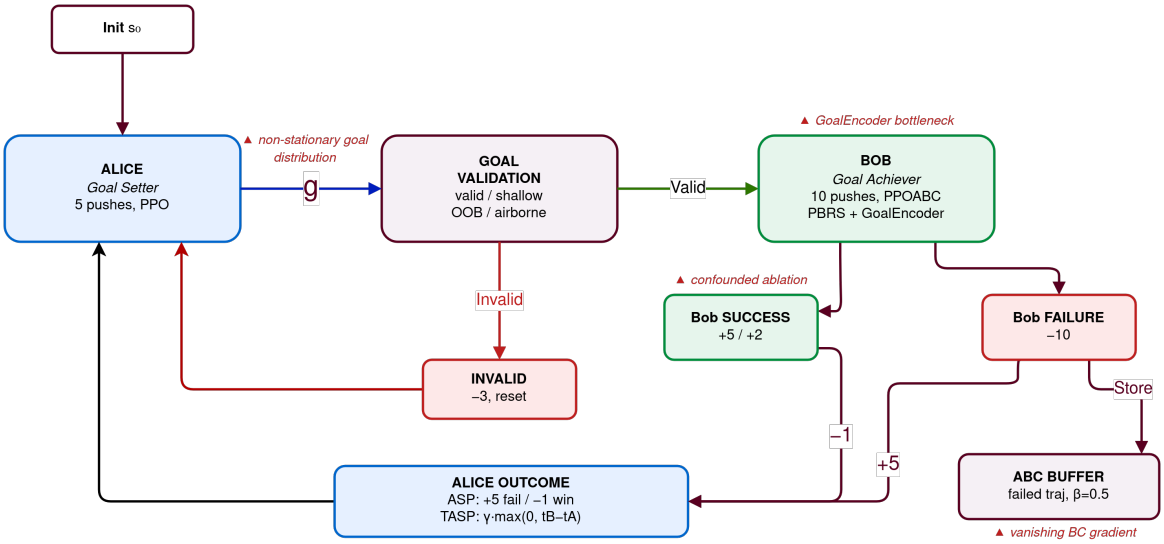

Figure 6: The asymmetric self-play loop. Alice proposes a goal by acting for a fixed number of pushes; the goal is validated; Bob then attempts it from a clean reset. Alice's reward differs between ASP-dPose (outcome-based) and TASP-dPose (time-based); Bob's SE(2) potential-based reward is the same in both.

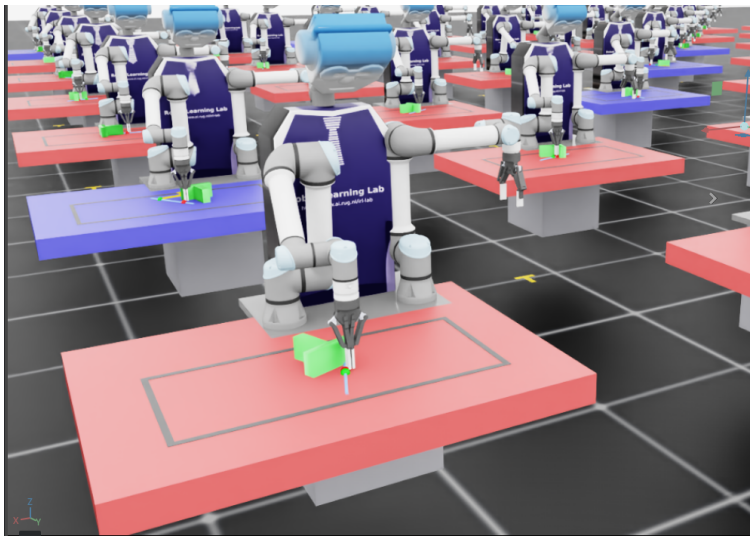

Figure 7: The self-play scene as rendered: Alice (red) displaces the T-block to the proposed goal; Bob (blue) attempts it from a clean reset.

Algorithm 4 states the self-play training loop shared by both variants; the disc and Bob-penalty models are the marked variations of it.

Algorithm 4 Asymmetric self-play training (ASP-dPose and TASP-dPose).

Require: Bob reward R_B (SE(2) potential-based); Alice reward mode; imitation weight β; Alice pushes H_A; Bob pushes H_B

initialise goal-setting policy π_A, goal-solving policy π_B with its goal encoder, and the value functions
for iteration = 1 to T **do**
  for each environment in parallel **do**
    reset the scene; Alice acts for H_A pushes and records trajectory τ_A
    $g \leftarrow$ object pose after Alice's pushes ▷ proposed goal
    if g is invalid (off the table or outside the workspace) **then**
      penalise Alice; skip Bob
    else
      reset the scene to the same start; Bob attempts g for up to H_B pushes, sampling $a \sim \pi_B(\cdot \mid s, g)$
      $t_B \leftarrow$ Bob's push count; record whether the combined gate is reached
      if Alice reward is outcome-based **then**
        $R_A \leftarrow$ reward when Bob fails, plus a valid-goal shaping bonus
      else
        $R_A \leftarrow \gamma_{sp} \max(0,\, t_B - t_A)$ ▷ time-based
      end if
      ▷ Bob-penalty variant additionally sets $R_B \leftarrow R_B - \gamma_{sp}\, t_B$
    end if
  end for
  update π_A with PPO on R_A
  update π_B with $\mathcal{L} = \mathcal{L}_{\mathrm{RL}} + \beta\, \mathcal{L}_{\mathrm{abc}}$, where $\mathcal{L}_{\mathrm{abc}}$ clones Alice's actions on failed goals
end for
return π_B

ASP-disc and TASP-disc. Two self-play models identical to ASP-dPose and TASP-dPose, except that the manipulated object is a rotationally symmetric disc rather than the T-block. A disc has no distinguished orientation, so its goal constrains position alone, and the combined success gate reduces to a position-only criterion. The mechanical reason a disc removes the orientation coupling that the T-block enforces is set out in Section 2.8 and Appendix A. These two models isolate the coupled multi-objective gate as a single variable. If self-play reaches the single-agent level on the disc yet underperforms it on the T-block, the coupled gate, rather than the reward or the goal encoder, is the factor that separates the two. Figure 8 shows the disc scene from above.

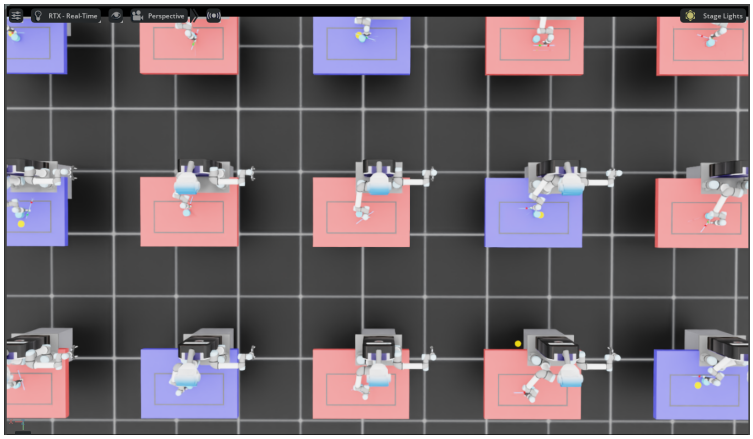

Figure 8: Bird's-eye view of the disc self-play scene; the goal constrains position alone.

Bob-penalty. A symmetric variant of time-based self-play in which the goal-solving agent is penalised for its own solve time, in addition to the goal-proposing agent being rewarded for it. It tests whether the asymmetry of the reward structure is necessary in this domain. Under this reward the model settles into a fail-fast equilibrium: failing immediately produces a higher return than attempting a longer solution, because the time penalty applies on every phase end, including catastrophic ones. Its held-out result is reported in Section 5.2.

Cross-environment counterparts. The single-agent, curriculum, and self-play models are retrained in a second, two-dimensional pushing simulator that uses neither inverse kinematics nor three-dimensional contact physics. These counterparts run on the central processing unit at environment counts of 64 and 256. Their role is to test whether the ordering of the methods holds in an environment of different fidelity, so that a conclusion does not rest on one simulator. The protocol is described in Section 4.4.

3.8 Why PBRS Assists Self-Play

Potential-based shaping improves self-play but does not make it competitive with single-agent training, and the reason is visible in the structure of the GAE advantage. The advantage estimate is a discounted sum of one-step temporal differences, each of which decomposes into a reward term and a value-bootstrap term:

$$\hat{A}_t = \sum_{l\geq 0}(\gamma\lambda)^l \underbrace{\left[\Phi(s_{t+l+1}) - \Phi(s_{t+l})\right]}_{\text{potential term, correctly signed}} + \sum_{l\geq 0}(\gamma\lambda)^l \underbrace{\left[\gamma V(s_{t+l+1}) - V(s_{t+l})\right]}_{\text{value term, biased when } V \text{ is stale}} \tag{22}$$

The potential term is correctly signed on every push, regardless of the accuracy of the value function, because the potential increases whenever the object moves toward the goal. The value term, by contrast, depends on the accuracy of V. In single-agent training the goal distribution is stationary, so V is accurate and both terms agree. Under self-play, Alice changes the goal distribution at every iteration, so V is perpetually estimated on a distribution that no longer holds, and the value term is biased. The potential term supplies a floor of correct learning signal, which lifts self-play above the level it reaches under the sparse or improvement reward. It cannot, however, remove the bias contributed by the stale value term. The consequence is that self-play learns a competence that is confined to the goal distribution Alice proposes, and the extent of that confinement, measured as the gap between the within-distribution scene success rate and the held-out scene success rate, is answered by the data in Chapter 5.

This account is a hypothesis about the learning dynamics. Two of its claims are measured directly through the training instrumentation described in Section 3.11, and the measurements are reported in Chapter 5. The first is the suppression of the imitation signal, measured as the fraction of demonstrated actions whose probability ratio falls outside the ABC clipping interval. The second is the within-distribution evaluation, which compares the within-distribution scene success rate, computed on the goals the proposer distributes, with the held-out scene success rate on the fixed evaluation suite. Whether the within-distribution competence transfers, and in which direction, is an empirical question answered by the data rather than assumed by the hypothesis.

3.9 Experiment-Set Overview

The nine models of Section 3.7 are the objects of study. A single training run of each, at one batch size and one seed, cannot support the statistical and causal claims this thesis makes. The evaluation is therefore organised as an experiment set of budget-matched training runs, summarised in Table 2. Its per-phase configurations and job counts are given in Chapter 4. This section states the two principles that make the experiment set a controlled study, together with the validation protocol and training instrumentation that every run shares. The numerical outcomes appear in Chapter 5.

The first principle is **multiple seeds with reported confidence intervals**. Single-seed reinforcement-learning results are known to be unreliable, because the variance between random seeds can exceed the difference between methods [53, 54]. Every headline comparison in this thesis is therefore repeated across several seeds. Each is reported as a mean with a 95 % confidence interval, so a claimed difference between two models is distinguished from seed noise.

The second principle is **budget matching**. The aim is to compare batch sizes without confounding them with the amount of experience. The total number of pushes seen over training is therefore held constant while the number of parallel environments is varied. The iteration count is set inversely proportional to the number of environments, so each configuration consumes the same total experience of about 23.8 million pushes. The temporal window of the recurrent policy is fixed at 15 pushes per rollout; only the number of parallel environments, and therefore the per-update batch size, changes. This isolates the per-update batch size, the quantity that drives the value-function bias of Section 3.8, from the total experience. Budget matching applies to the Isaac Lab runs; the second-simulator runs instead use a fixed iteration count, as described in Chapter 4. To examine whether the single-GPU budget is itself the cause of the self-play result, the two self-play subjects are additionally trained at 256 and 2,048 environments, so that compute scale is tested rather than assumed.

Two conventions hold across every self-play run in the experiment set. The imitation term is enabled in the canonical self-play models, and disabled only for the ablation arm. Both the single-agent and the self-play validators sample actions from the policy, rather than taking the most probable action, so the two families are compared under one protocol.

Phase	Question	Design
Baseline and efficiency	Is the goal-reaching task solvable, and how few environments does the potential-based reward need?	PPO-Baseline and PPO-PBRS across four environment counts, three seeds each
Seed intervals	Is each model difference larger than seed noise?	the baseline, the potential-based agent, the curriculum, and both self-play subjects at the baseline budget, five seeds
Coupling isolation	Does self-play reach the baseline level when the goal is position-only?	the two disc self-play models against the two T-block subjects, five seeds
Self-play scale	Does more compute rescue self-play, or is scale not the cause?	both self-play subjects at 256 and 2,048 environments, three seeds
Component ablations	Which component of the self-play package matters?	the self-play subjects with the imitation term, then the goal encoder, removed one at a time, three seeds
Imitation positive control	Does imitation help when the task is easy?	disc self-play with the imitation term enabled and disabled, three seeds
Reward structure	Does the symmetric reward collapse the loop?	the Bob-penalty model at the baseline budget, five seeds
Cross-environment	Does the ordering hold in a second simulator?	the single-agent model at 64 and 256 environments with push lengths 15 and 45; the curriculum model at 64 environments with both push lengths; the self-play model at 64 and 256 environments with push length 15; one seed each

Table 2: The experiment set. Each phase answers one question; the per-phase training configurations, environment counts, seeds, and job counts are tabulated in Chapter 4.

3.10 Validation Protocol

Every trained policy is evaluated on the same held-out suite of 30 T-block scenes. The suite, the evaluation budget, the success measures, and the seed aggregation are defined once, in the validation protocol of Section 4.3 in the next chapter.

Three protocol decisions make the numbers comparable across models. Each corrects a subtle way the comparison could otherwise be invalidated. First, the checkpoint selected for validation is the one that maximised the *combined* position-and-orientation success rate during training, not the looser single-distance threshold used internally by the self-play reward. Selecting on a different criterion than the one reported would flatter the self-play models. Second, the self-play policies trained on the rotationally symmetric disc are validated on disc scenes, whose goal orientation is unconstrained, not on the T-block scenes they never trained on. Validating an agent on a task it never trained for would void its result. Third, checkpoints that lack a combined-gate best model, and are validated only from their latest checkpoint, are recorded separately and excluded from the confidence-interval aggregation. A partially-trained run therefore cannot

contaminate a headline mean. The per-seed results are aggregated into a mean with a 95 % confidence interval per model, batch size, and iteration count.

3.11 Training Instrumentation for the Self-Play Mechanism

The account in Section 3.8 of why potential-based shaping only partially assists self-play rests on two claims: that the value-function term of the advantage estimate is biased under the shifting goal distribution, and that the behavioural-cloning signal is suppressed when the goal-proposing agent's demonstrated actions lie far from the goal-solving agent's policy. Only the second claim is measured directly, and the measurement is reported in Chapter 5.

The value-function claim remains a hypothesis. The logged value error is the mean absolute difference between the value estimate and the empirical return, an absolute quantity that scales with the magnitude of the return itself. The single-agent and self-play rewards differ in scale, so a near-zero error under self-play reflects a near-zero return rather than an accurate value function, and the measurement cannot separate a stale value function from a small one.

For the behavioural-cloning claim, the imitation term is instrumented in three ways: the share of demonstrated actions whose probability ratio falls outside the clipping interval, the mean ratio, and the mean probability the goal-solving agent assigns to the goal-proposing agent's demonstrated actions. The clipping mechanism inherited from the imitation loss of [9] zeroes the gradient for any demonstrated action whose ratio leaves the trust interval. A large clipped fraction is therefore the direct signature of a suppressed imitation signal. Measuring this fraction converts the clip-saturation account into a reported quantity. A positive control on the rotationally symmetric disc, where behavioural cloning is enabled and the task is easy, tests whether the mechanism can help at all in this setting. If imitation assists anywhere it should assist there, so its failure on the coupled T-block task cannot then be dismissed as a broken implementation.

4 Experimental Setup

This chapter describes how the nine models of Chapter 3 are trained and evaluated. It specifies the tools and hardware, the training configuration and the experiment set, and the held-out validation protocol. It then describes the cross-environment comparison against a second simulator, an off-policy baseline based on Soft Actor-Critic, and the metrics reported in the results.

4.1 Tools and Infrastructure

Training and Isaac-Lab evaluation run on a single RTX Pro 6000 GPU with 96 GB of memory. The software stack is Isaac Sim 5.1.0, Isaac Lab 2.3.0, cuRobo 0.7.5 for batched Inverse Kinematics (IK), and PyTorch 2.7.0. A second environment, the two-dimensional `gym-pusht` simulator, is used for the cross-environment comparison of Section 4.4. It runs on 32 CPU cores and uses neither IK nor three-dimensional contact physics. The off-policy baseline of Section 4.5 uses Stable-Baselines3 2.7.0. Running the same models in two simulators of different fidelity separates conclusions that hold across environments from artefacts of a single implementation.

4.2 Training Configuration and the Experiment Set

At the 528-environment baseline, every model is trained for approximately 3,000 iterations. This corresponds to about 1.6 days of wall-clock time per model on the RTX Pro 6000. At 15 pushes per rollout it yields a batch of roughly 7,920 push transitions per PPO update. For the reward-efficiency comparison, the improvement-reward baseline and the potential-based agent are trained across the same environment counts, from 256 to 2,048, so the two rewards are measured at matched parallelism rather than at a single batch. The two self-play subjects are additionally trained at 256 and 2,048 environments, which tests whether the single-GPU compute budget is the cause of their result. The PPO hyperparameters are a learning rate of 3×10^{-4}, a clipping range of 0.2, a discount factor of 0.998, a GAE parameter of 0.95, and an entropy coefficient of 0.005. The self-play models additionally use Alice Behavioral Cloning at $\beta = 0.5$.

Two points about the training history are stated for reproducibility. First, all reported checkpoints post-date a sequence of corrections to the reward and the environment. The reported numbers are therefore produced on the corrected code, and earlier runs are excluded. One seed of ASP-dPose stopped at iteration 34 and is likewise excluded from the training-curve figures, though its checkpoint remains part of the held-out evaluation. The held-out validation for one seed each of TASP-dPose and Bob-penalty is still running at the time of writing, so both models are reported on the four seeds whose validation has completed. Second, the improvement-reward baseline and the potential-based agent are compared across a shared range of environment counts, so that any difference between the two rewards cannot be attributed to unequal parallelism. The budgets are tabulated in Appendix H.

Every Isaac Lab run in the experiment set is budget-matched. The total experience is held at about 23.8 million pushes, and the iteration count is set inversely proportional to the number of parallel environments, so a change in the number of environments changes the per-update batch size alone. This design has one confound that is stated wherever a scale result appears: a larger batch also means fewer gradient updates over training, so batch size and update count move together and cannot be fully separated. Table 3 lists the phases, the models each one trains, the environment counts, the matched iteration counts, and the seeds.

Phase	Models	Envs	Iterations	Seeds	Runs
Baseline and efficiency	PPO-Baseline, PPO-PBRS	256–2,048	773–6,188	3	24
Seed intervals	Baseline, PBRS, Curriculum, ASP-dPose, TASP-dPose	528	3,000	5	25
Coupling isolation	ASP-disc, TASP-disc	528	3,000	5	10
Self-play scale	ASP-dPose, TASP-dPose	256, 2,048	6,188; 773	3	12
Component ablations	ASP-dPose, TASP-dPose	528	3,000	3	12
Imitation positive control	ASP-disc	528	3,000	3	6
Reward structure	Bob-penalty	528	3,000	5	5
Cross-environment	single agent, curriculum, self-play	64, 256	3,000	1	8

Table 3: The phases of the experiment set. Environments and iterations trade off to hold the total experience at about 23.8 million pushes for the Isaac Lab runs; the cross-environment runs instead use a fixed 3,000 iterations and a single seed, and vary the per-update batch through the environment count and the push length. The experiment set comprises 94 training runs in Isaac Lab and 8 in the second simulator.

4.3 Validation Protocol

Every model is evaluated on the same held-out suite of 30 T-block scenes, illustrated in Figure 9. The suite is divided into three groups of ten. The first ten are rotation-dominant scenes with large orientation changes. The next ten are position-only scenes whose goal orientation equals the start orientation. The last ten are combined scenes that require both a position and an orientation change. Within each group the difficulty ranges from easy to hard, graded by the start-to-goal distance and the magnitude of the required rotation.

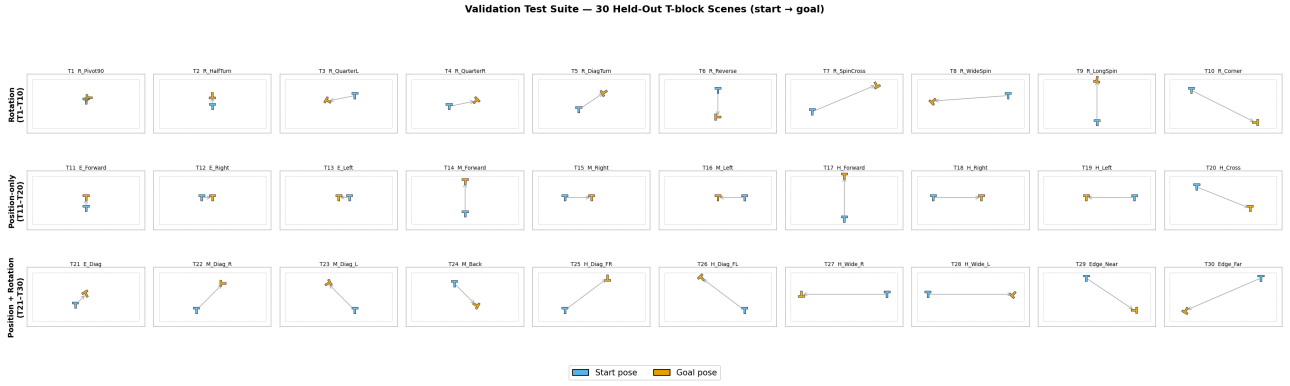

Figure 9: The 30-scene held-out validation suite. Each panel shows the block start pose and the goal pose for one scene. The suite is identical for every model.

Each scene is attempted 20 times, and within each attempt the policy is allowed up to 30 pushes. Both the single-agent and the self-play validators sample actions from the policy, rather than taking the most probable action, so every model is scored under one protocol. Two success measures are reported and kept distinct throughout. The *scene success rate* counts a scene as solved if at least one of its 20 attempts satisfies the combined gate of Section 3.2. This

is the headline number, and follows the convention common in manipulation evaluation. The *trial success rate* is the fraction of individual attempts that satisfy the gate, and is always lower than the scene success rate. Reporting both prevents the scene protocol from overstating performance, and the gap between them is stated wherever the headline number appears. Each model is evaluated across its trained seeds, and the results are aggregated into a mean with a 95 % confidence interval per model, batch size, and iteration count.

A parallel within-distribution evaluation applies the same scene protocol to goals drawn from the model's own training distribution rather than from the fixed 30-scene suite. For the self-play models the frozen goal-proposing agent generates each goal by acting for its proposal budget from a sampled start pose. The goal is retained only if it passes the validity check of Section 3.7. The frozen goal-solving agent then attempts it under the same per-goal budget as the held-out suite. For the single-agent models the goals are sampled from the training goal distribution. The within-distribution scene success rate is the fraction of these goals solved in at least one attempt, aggregated across the trained seeds with a 95 % confidence interval. It is reported alongside the held-out scene success rate wherever the generalisation gap of Section 5.4 is at issue.

Every held-out number reported in this thesis comes from this full evaluation: all 20 attempts per scene with up to 30 pushes per attempt, across every trained seed of every model. No abbreviated pass is reported.

4.4 Cross-Environment Protocol

To test whether the ordering of the models is a property of the methods rather than of Isaac Lab, the single-agent, curriculum, and self-play models are retrained and evaluated in `gym-pusht` under the same 30-scene gate, with 20 attempts per scene. The two-dimensional counterpart is shown in Figure 11. The two environments differ mainly in the size of the PPO batch. Isaac Lab produces about 7,920 transitions per update at the 528-environment baseline, while `gym-pusht` produces between about 960 and 11,520, depending on the environment count and the push length. The gym runs use a fixed 3,000 iterations and sweep the per-update batch through these two settings, so the comparison tests whether the ordering of the methods survives a second, lower-fidelity simulator, rather than matching the total experience between the two environments. The single-agent counterpart covers all four combinations of 64 and 256 environments with 15 and 45 pushes per rollout. The curriculum counterpart runs at 64 environments with both push lengths, and the self-play counterpart at 64 and 256 environments with 15 pushes. The held-out numbers reported in the results come from the primary configuration, 64 environments with 15 pushes per rollout; the remaining configurations are training-only. Figure 10 summarises the throughput and batch size of the two environments.

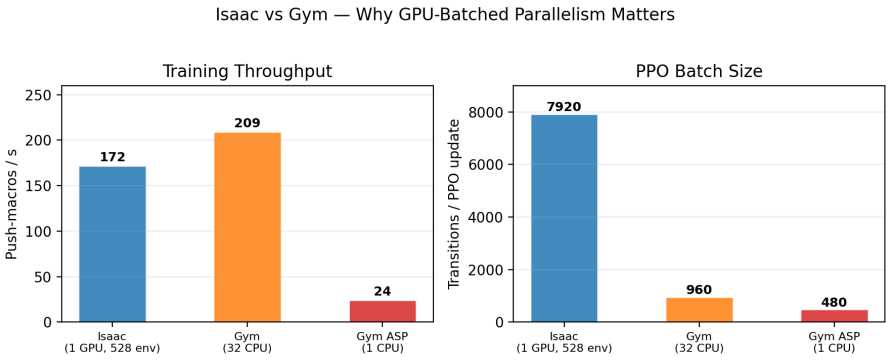

Figure 10: Throughput and PPO batch size for Isaac Lab and `gym-pusht`. Raw throughput is comparable; the per-update batch differs by roughly a factor of eight, which is the variable the cross-environment comparison isolates.

Figure 11: The two-dimensional `gym-pusht` counterpart of the pushing task.

4.5 Off-Policy Baseline: SAC with Hindsight Experience Replay

The principal models are all on-policy. To provide an external point of calibration from a different algorithm family, an off-policy baseline is built from Soft Actor-Critic (SAC) [16] combined with Hindsight Experience Replay (HER) [18]. SAC is a natural counterpart to PPO for this problem. It is sample-efficient through experience replay, and its maximum-entropy objective (Equation 9) supports exploration under sparse reward. HER addresses the sparse goal-conditioned setting directly: it relabels the object pose actually reached in a failed attempt as if it had been the goal, so every attempt yields usable transitions.

The baseline runs on a direct control environment through Stable-Baselines3, and reuses the shared push primitive and the cuRobo IK pipeline of Section 3.2. Two adaptations are required. First, SAC is a continuous-action algorithm, so the four push parameters become a continuous action in $[-1, 1]^4$ and are decoded to the same object-relative primitive, rather than the MultiCategorical variable used by the PPO models. Second, the goal-conditioned observation is expressed in the dictionary form the replay buffer expects, with separate observation, achieved-goal, and desired-goal entries. The baseline is trained and evaluated under the same 30-scene

protocol as the principal models, so its success rate is directly comparable. Its role is calibration rather than competition. It indicates whether an off-policy, replay-based method reaches performance similar to the on-policy potential-based model. The baseline was attempted but did not converge within the available compute budget. That status is final and is reported with the results; the published FetchPush result of [18] is used as plausibility calibration in its place, as stated in Section 5.6.

4.6 Metrics

The results report three success rates, each with its sample size and protocol stated at the point of use. The *held-out scene success rate* is the outcome of the fixed 30-scene evaluation of Section 4.3, and is the measure of generalisation this thesis reports as its headline. The *within-distribution scene success rate* applies the same scene protocol to goals drawn from the model's own training distribution. For the self-play models these goals are proposed by the frozen goal-proposing agent and accepted by the validity check; for the single-agent models they are sampled from the training goal distribution. The measure captures competence on the goals the model was trained to solve, and its gap against the held-out rate is the quantity of interest in Section 5.4. The *training-time success rate* is a per-push rate on the combined gate, the median of the last five training iterations, averaged across seeds. It is retained as an auxiliary diagnostic only and does not enter the headline comparisons.

The breakdown by scene type, position-only against position-and-rotation, is reported because it localises where each model fails. Mean position error and mean orientation error are reported as continuous measures of near-misses. For the self-play models the training logs also provide the per-axis success rates: the fraction of pushes that satisfy the position gate alone and the rotation gate alone. These are compared against the combined-gate rate and against the held-out result to diagnose where the within-distribution competence is confined. Average push count per attempt is reported as a measure of efficiency. One further quantity is logged during self-play training and reported in the results: the share of demonstrated actions the imitation term clips away (the ABC clip fraction), a direct measure of how strongly the imitation signal is suppressed. All reported numbers trace to the experiment set recorded in the implementation log.

5 Results

This chapter reports the generalisation of every model to the held-out 30-scene suite, organised by research question. Section 5.1 establishes that the task is solvable. Section 5.2 addresses whether asymmetric self-play succeeds under the single-GPU budget (RQ1). Section 5.3 isolates the setup dimension that separates success from failure (RQ2). Section 5.4 identifies the underlying mechanism (RQ3). Section 5.5 reports the second-simulator result.
Three success measures are reported and kept distinct throughout, and all are defined in Section 4.6. The *held-out scene success rate* is computed on the fixed 30-scene suite that no model saw during training, and is the measure of generalisation this thesis reports as its headline. The *within-distribution scene success rate* is computed on goals drawn from the model's own training distribution, and its gap against the held-out rate is the mechanism of Section 5.4. The *training-time success rate* is an auxiliary per-push rate computed on the goals the models trained on. Unless stated otherwise, headline models report 5 seeds at 528 environments; ablations and scale sweeps report 3 seeds; confidence intervals are 95 % across seeds. The held-out numbers in this chapter follow the evaluation protocol defined in Section 4.3, and every comparison reported here is already decisive under that protocol. Table 4 collects the principal numbers.

Model	**Scene SR**	**Trial SR**	**Pos-only**	**Seeds**
PBRS (potential-based)	**79.3 % ± 3.5 %**	70.2 %	100 %	5
Curriculum	**80.0 % ± 4.1 %**	70.2 %	100 %	5
ASP-disc (position-only)	7.3 % ± 1.8 %	5.8 %	—	5
TASP-disc	6.7 % ± 0.0 %	4.9 %	—	5
ASP-dPose (outcome)	0.0 %	0.0 %	0 %	5
TASP-dPose (time-based)	0.0 %	0.0 %	0 %	4
Bob-penalty	0.0 %	0.0 %	0 %	4

Table 4: Held-out generalisation of the headline models at 528 environments. Scene SR counts a scene solved if any of its attempts passes the combined gate; trial SR is the per-attempt rate. Pos-only is the scene SR on the ten position-only scenes. Values are seed means with 95 % confidence intervals across seeds, under the evaluation protocol of Section 4.3.

5.1 Baseline

The single-agent models establish that the task is solvable, which is the precondition for interpreting any self-play result as a property of the method rather than the task. PBRS reaches a held-out scene success rate of 79.3 % (± 3.5 percentage points, 5 seeds) at 528 environments, and the curriculum model reaches 80.0 % (± 4.1 pp, 5 seeds). On the ten position-only scenes both reach 100 %; on the twenty scenes that also require an orientation change they reach 69 % and 70 %. The single-agent models therefore fail only on rotation-requiring scenes, and never because of position. Their trial success rate of 70.2 % shows that the policy is reliable as well as capable: a scene is solved in the majority of attempts, not in a single lucky one. The per-scene errors of Figure 12 show where the remaining failures concentrate: position error stays below tolerance on every scene, and the unsolved scenes are those whose orientation error exceeds the 0.2 rad threshold.

The single-agent models show no generalisation gap. Their training goals are sampled uniformly over the workspace, so the within-distribution evaluation of Section 4.6 draws goals from the same distribution as the held-out suite, and the two success rates coincide. The 79.3 % held-out result is therefore also the single-agent's within-distribution competence. This distinguishes the single-agent models from the self-play models of Section 5.4, whose within-distribution success collapses on the held-out suite.

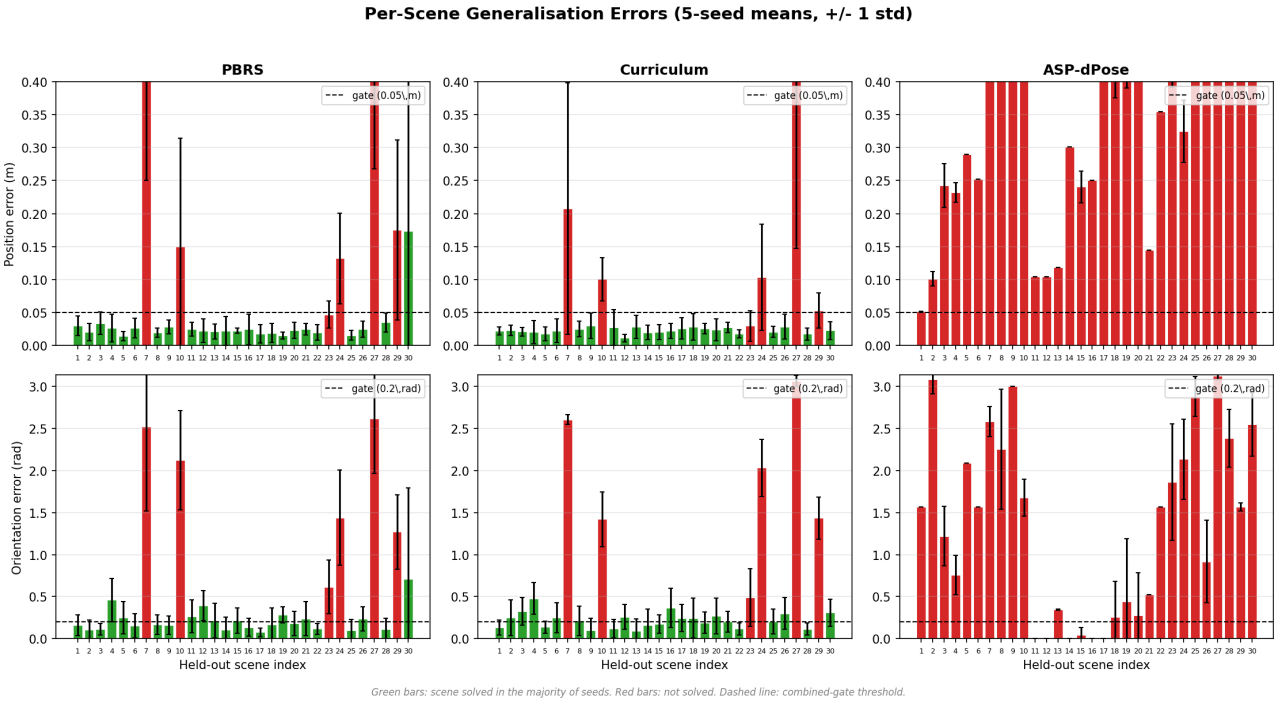

Figure 12: Per-scene generalisation errors, seed means across 5 seeds (± 1 standard deviation). Top row: position error; bottom row: orientation error. Columns: PBRS, Curriculum, and ASP-dPose. Green bars mark scenes solved in the majority of seeds; the dashed line marks the combined-gate threshold. The single-agent models leave only rotation-dominant scenes unsolved, while ASP-dPose solves none.

5.2 Research Question 1: ASP performance under the single-GPU budget

The self-play models do not generalise to the held-out suite at all. ASP-dPose and TASP-dPose each solve zero of the 30 scenes, in every seed (5 and 4 seeds respectively). The Bob-penalty model is likewise at zero. This is not a marginal underperformance against the 79 % of the single-agent model; it is a complete failure to transfer.

The two Alice reward formulations are indistinguishable at this level. The outcome-based reward of [9] and the time-based reward of [8] both produce zero held-out successes, so the reward-formulation dimension cannot be resolved on the T-block: both variants fail entirely. The same holds for the symmetric Bob-penalty variant. The failure is one of transfer, not of learning. On goals the goal-proposing agent itself produces, the self-play policy solves 61.4 % of disc scenes and 11.7 % of T-block scenes (5 seeds each, Section 5.4), and none of that competence reaches the held-out suite. The comparisons that can be made at this level are between the

within-distribution behaviour of Section 5.4 and the held-out result, where a consistent picture emerges.
The disc models, whose goal constrains position alone, are marginally above zero. ASP-disc solves 7.3 % of its 30 disc scenes (± 1.8 pp, 5 seeds) and TASP-disc 6.7 % (± 0.0 pp, 5 seeds). The couple of scenes each disc model solves are the easiest of the suite. This places the disc contrast of Section 5.3.1 in its proper light: self-play fails on both objects, and fails completely on the coupled-goal T-block.

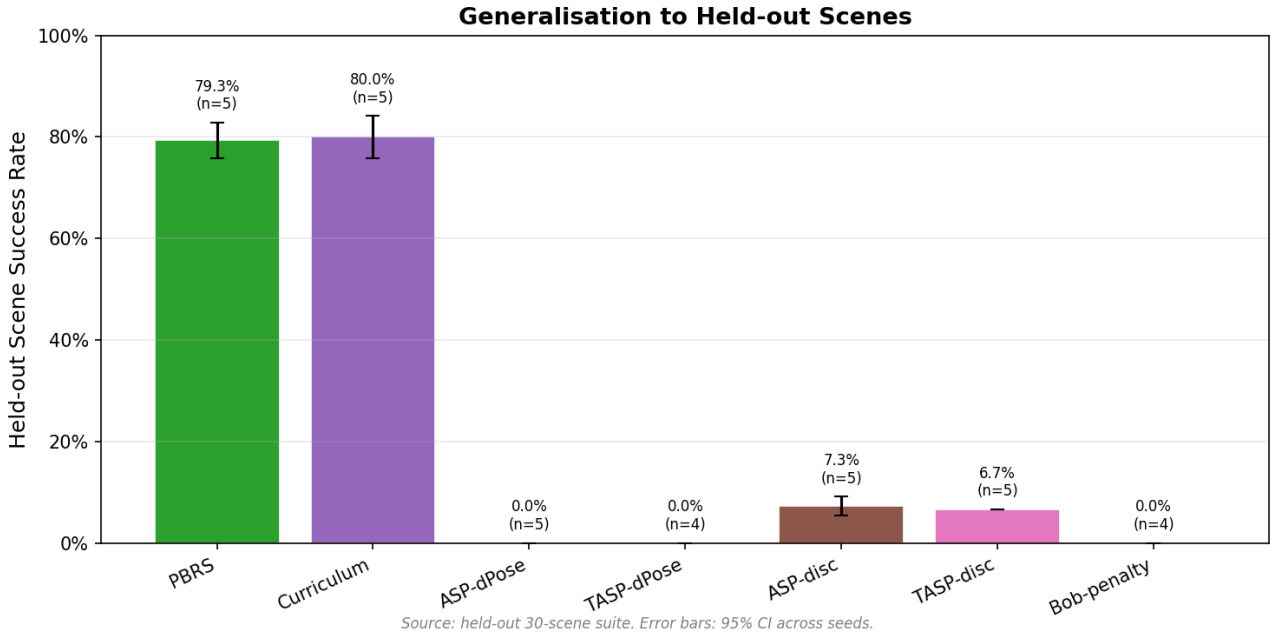

Figure 13: Held-out scene success rates for the headline models at 528 environments. The single-agent models solve roughly four scenes in five; the disc self-play models solve fewer than one in ten; the T-block self-play models solve none.

5.3 Research Question 2: Setup dimensions

This section reports the single-dimension isolations that decompose the self-play result. Each isolates one factor that differs between the successful prior settings and the present setup.

5.3.1 Goal Coupling

The disc self-play models isolate the coupled goal as a single variable. On the rotationally symmetric disc, whose goal constrains position alone, self-play solves 7.3 % of held-out scenes. On the T-block, whose goal couples position and orientation, the same mechanism solves 0.0 %. The pose, the push primitive, the Bob reward, and the self-play loop are identical between the two variants; only the object and the success criterion differ.
The within-distribution evaluation of Section 5.4 measures the coupling directly. On goals the goal-proposing agent itself generates, the disc model solves 61.4 % of scenes (± 2.1 pp, 5 seeds) while the T-block model solves 11.7 % (± 1.5 pp, 5 seeds). The coupled gate therefore reduces within-distribution competence by roughly 50 percentage points before any transfer is required. The held-out suite then collapses both rates, the disc to 7.3 % and the T-block to zero. The failure thus separates into two effects of comparable size. The coupled gate accounts for the

within-distribution drop from 61.4 % to 11.7 %; the generalisation collapse accounts for the drop from each within-distribution rate to its held-out rate, roughly 54 percentage points on the disc. Neither effect alone explains the full result, and the second is the subject of Section 5.4.

The single-agent models show the same gate effect in a solvable setting. On the ten position-only scenes they reach 100 %; on the twenty scenes that also demand an orientation change they reach 69–70 %. The coupled gate is hard even for the model that solves the task, which is why it compounds self-play's generalisation failure decisively.

5.3.2 Reward Formulation

The two Alice reward formulations produce identical held-out outcomes: zero scenes solved for both ASP-dPose and TASP-dPose. The self-regulating property of the time-based reward of [8] does not confer a measurable advantage in this setup, and neither reward transfers. The reward-formulation dimension is therefore not the factor that separates the successful prior settings from this one: both formulations fail equally here.

5.3.3 Compute Scale

The two self-play subjects were trained at 256, 528, and 2,048 parallel environments with matched total experience, so that the per-update batch size varies while the total pushes stay fixed. On the held-out suite ASP-dPose solves 0.0 % of scenes at 256 and 528 environments (3 and 5 seeds), and 1.1 % (± 4.8 pp, 3 seeds) at 2,048, where a single seed solves a single scene. TASP-dPose solves 0.0 % at all three counts. An eightfold increase in the parallel batch does not lift the self-play result above the noise floor. Compute scale is therefore ruled out as the cause of the underperformance, in the strongest sense available: the curve is not merely flat, it is flat at zero.

5.3.4 Imitation Signal

The imitation term introduced by [9], Alice Behavioural Cloning at $\beta = 0.5$, is instrumented through the fraction of demonstrated actions whose probability ratio falls outside the clipping interval. For every ABC-enabled model this clip fraction is **0.000** at every training iteration: the goal-proposing agent's exploratory pushes lie so far from the goal-solving agent's policy that the clipping zeroes the gradient, and the imitation signal never flows.

The ablation shows the signal's absence has no measurable effect because there is nothing left to preserve. Removing ABC changes the held-out scene SR of ASP-dPose from 0.0 % to 0.0 % (3 seeds), and the same holds for TASP-dPose and for the disc models (6.7 % with and without). The positive control on the disc, where the task is easiest and the imitation term would be most likely to help, also shows no effect. The practical consequence is that the imitation term contributes nothing in this discrete macro-action domain; the clip-saturation measurement explains why.

5.3.5 Goal Representation

Removing the GoalEncoder from Bob and providing the full object-and-goal observation directly to the policy encoder changes the held-out scene SR of ASP-dPose from 0.0 % to 0.0 % (3

seeds). The eight-dimensional latent compression is not the factor that limits self-play, consistent with the disc isolation, where the same encoder is present and self-play retains its marginal generalisation.

5.4 Research Question 3: Mechanism

The mechanism of the self-play failure is the gap between the within-distribution competence and the held-out performance. Table 5 and Figure 14 place the two measures side by side.

Model	Within-dist SR	Held-out SR	Seeds
ASP-disc	61.4 % ± 2.1 %	7.3 %	5
ASP-dPose	11.7 % ± 1.5 %	0.0 %	5

Table 5: Within-distribution against held-out scene success for the two self-play subjects. Within-distribution SR is the scene success rate on goals proposed by the frozen goal-proposing agent and accepted by the validity check, under the evaluation protocol of Section 4.3; held-out SR is the 30-scene rate. Values are seed means with 95 % confidence intervals across the seeds listed. The single-agent models are omitted because their uniform training goal distribution coincides with the held-out suite, so their within-distribution and held-out rates are the same.

The within-distribution evaluation separates the two effects that the headline numbers confound. On goals the goal-proposing agent itself produces, the disc model solves 61.4 % of scenes and the T-block model 11.7 %, across five seeds each. The same models solve 7.3 % and 0.0 % of held-out scenes. The single-agent models show no such separation: their uniform training goal distribution coincides with the held-out suite, so their within-distribution and held-out rates agree at roughly 79 %.

Two effects of comparable size are now measurable. The first is the coupled gate. Within the proposer's own distribution, replacing the position-only disc goal with the coupled T-block goal reduces scene success from 61.4 % to 11.7 %, a drop of roughly 50 percentage points. The second is the generalisation collapse. The disc model loses a further 54 percentage points between its within-distribution rate and the held-out suite, and the T-block model falls from 11.7 % to zero. The coupled gate therefore bounds competence on the proposer's own goals, while the narrow goal distribution prevents that competence from transferring.

The per-axis training logs confirm that the coupled gate, not a missing skill, limits the T-block. ASP-dPose satisfies the position gate on 41.0 % of its training pushes and the rotation gate on 33.9 %, yet the combined gate is met on only 11.7 % of within-distribution scenes. The two skills are present but rarely co-satisfied. On the disc, where only position is demanded, the within-distribution rate rises to 61.4 %, which confirms that the coupling is what suppresses the T-block within-distribution. The residual transfer failure, in which even the disc's competence does not reach the held-out suite, is a property of the goal distribution itself. Alice proposes goals near her own final pose, so the policy is trained on goals the held-out suite never presents, and its competence is tied to those goals rather than to a transferable skill.

This answers RQ3. Self-play does not fail because a component of the method is broken, and it does not fail from a single cause. Two effects compound: the coupled multi-objective gate reduces the competence the policy acquires on its own goals by roughly 50 percentage points, and the narrow goal distribution prevents that competence from reaching the held-out suite.

Both follow from the same curriculum, and together they reduce the disc's 61.4 % to a held-out 7.3 % and the T-block's 11.7 % to zero.

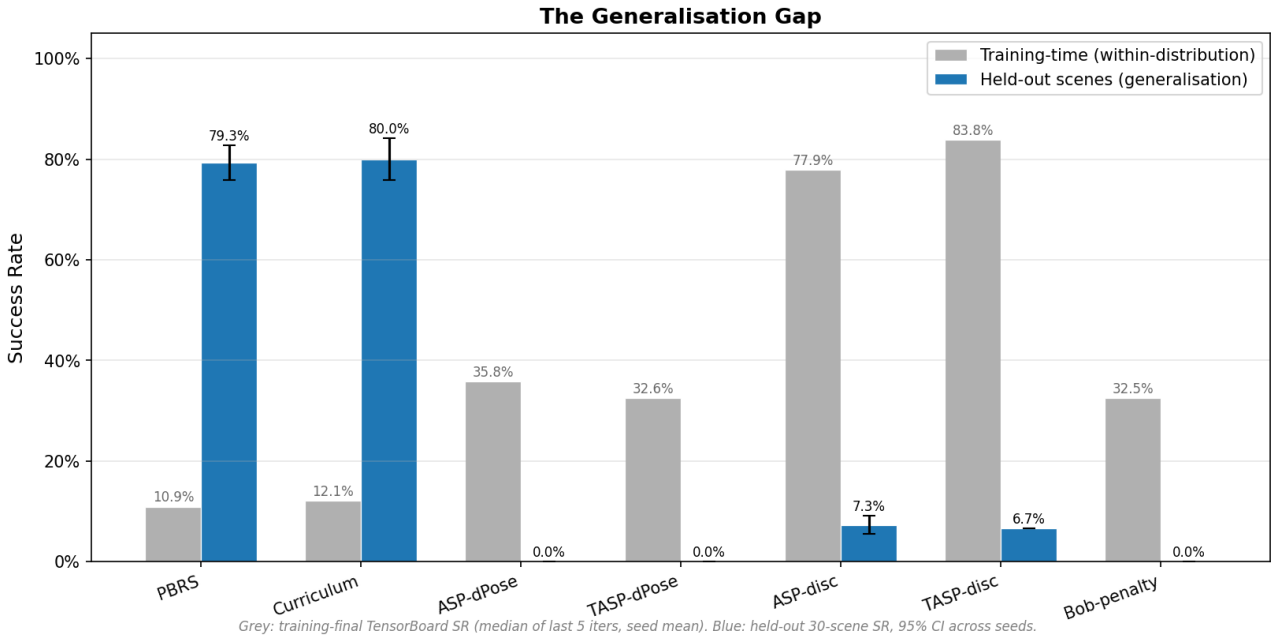

Figure 14: The generalisation gap. Grey bars: within-distribution scene success rate (seed mean). Blue bars: held-out scene success rate with 95 % confidence intervals. The single-agent models show no gap, because their training goal distribution coincides with the held-out suite; the self-play models collapse from their within-distribution rate to the held-out rate.

5.5 Cross-Environment

The second-simulator counterpart provides a single-seed qualitative check, and each number below rests on that one seed. The numbers come from the primary configuration of 64 environments and 15 pushes per rollout; the remaining configurations of Section 4.4 are training-only. The single-agent model solves 3.3 % of the 30 scenes, far below the 79.3 % of the Isaac Lab model. The curriculum counterpart solves 9.0 %, and the self-play counterpart solves 1.0 %. The lower absolute rates reflect the lower fidelity of the second simulator and its single seed, not a contradiction of the Isaac Lab result. What transfers is the ordering. The self-play model remains the weakest of the three in both simulators. The curriculum model matches or exceeds the plain single-agent model, consistent with the curriculum comparison of Chapter 6.

5.6 Qualitative Behaviour

Figure 15 shows keyframes of the single-agent PBRS policy solving three held-out scenes. The policy approaches the block along the object-relative offset, pushes along the decoded direction, and repeats until the combined gate is met. Its failures, on the rotation-dominant scenes, show position under control and orientation outside the 0.2 rad tolerance, which matches the per-type breakdown of Table 4.

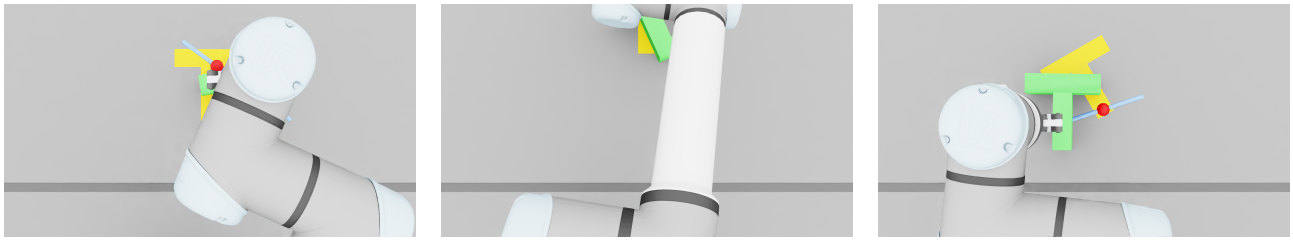

Figure 15: Keyframes of the single-agent PBRS policy on three held-out scenes.

The off-policy baseline, SAC with Hindsight Experience Replay, did not converge within the available compute budget. Its status is recorded so that the on-policy results are read against a concrete off-policy attempt rather than in isolation. The failure is not evidence against the algorithm family: DDPG with hindsight experience replay reaches a median success rate of 100 % on FetchPush, a simulated tabletop pushing task, in [18]. That published result serves as plausibility calibration only. The simulator, the object, the action space, and the compute budget all differ from the present task, so no success rate is compared directly.

6 Discussion and Conclusion

This chapter interprets the results, states the contributions against the research questions, records the limitations of the study, and lists directions for future work.

6.1 Interpretation

Unless stated otherwise, every quantitative claim in this section refers to the seed means reported in Table 4 of Chapter 5; claims that rest on 3 seeds are marked as such.

The headline finding is a complete failure of generalisation. The single-agent models solve 79–80 % of the held-out scenes, across five seeds each, and solve every position-only scene. The self-play models on the same T-block task solve zero scenes, in every seed, under every reward formulation, at every environment count from 256 to 2,048. The failure is not a matter of degree. It is a categorical difference between two training setups applied to the same task.

The regime analysis of Section 5.3 locates the cause. Three candidate dimensions are ruled out by direct experiment. Compute scale is ruled out by a sweep that is flat at zero across an eightfold increase in the parallel batch. The imitation signal is ruled out as a cause because the clip-saturation measurement shows it never flows: the clip fraction is 0.000 at every training iteration in the discrete macro-action space, and removing the term changes nothing. The goal representation is ruled out by an ablation that leaves the outcome unchanged. The dimension that does matter is the coupled multi-objective gate, whose contribution the within-distribution evaluation now measures directly: it reduces success on the proposer's own goals from 61.4 % on the disc to 11.7 % on the T-block.

That collapse is the mechanism of Section 5.4, now measured directly rather than inferred. On goals the goal-proposing agent itself produces, the self-play policy solves 61.4 % of disc scenes and 11.7 % of T-block scenes (5 seeds each); on the held-out suite those rates fall to 7.3 % and 0.0 %. The within-distribution evaluation separates two effects of comparable size. The coupled gate reduces within-distribution success from 61.4 % on the disc to 11.7 % on the T-block. The narrow goal distribution then prevents transfer, and the disc collapses by a further 54 percentage points to its held-out rate. The single-agent models show no such gap, because their uniform training goals cover the suite. The generalisation gap therefore reverses between the two setups, which is the quantitative signature of the finding.

This reframes the mechanism with respect to the provisional account given earlier. The self-play failure is not the failure of any single component of the method, and it is not a pure composition failure. Two effects act together. Asymmetric self-play derives its curriculum from the goal-proposing agent's own reachable region of the workspace. In contact-rich pushing with discrete macro-actions that region is narrower than the evaluation distribution, and the competence acquired there does not transfer to held-out scenes. The coupled gate bounds that competence before transfer is even required.

6.2 Relation to Prior Work

The findings agree with the theory of the papers this thesis builds on, while qualifying the empirical claims of the papers it tests.

On reward shaping, the result confirms [6] under conditions the original theorem did not examine. The potential-based reward produces a single-agent policy that solves 79 % of held-out

scenes, and the policy-invariance guarantee holds under stochastic PhysX contact. The demonstration that this guarantee survives contact-rich physics is the supporting contribution of this thesis.

On asymmetric self-play, the result scopes the method rather than refuting it. [8] demonstrated self-play in reversible gridworlds, where the goal-proposing agent's own trajectory is a valid solution to the goal it sets by construction. [9] scaled it to robotic manipulation with distributed compute and continuous actions. The present setup differs on the dimension that matters here: the goal distribution the proposer can produce is narrower than the distribution the evaluation demands, so the by-construction achievability of training goals does not extend to held-out scenes. The within-distribution evaluation shows self-play retains substantial position competence (61.4 %) when the gate is position-only, and the coupled gate reduces that to 11.7 % on the T-block before transfer is even required. This is a generalisation-bound result with a coupled-gate component, and it is specific to this setup.

The ABC clip-saturation finding connects a design choice in [9] to an outcome observed here. The PPO-style clipping on the behavioural-cloning loss prevents aggressive updates in continuous-action domains where the two agents' action distributions overlap. In the discrete macro-action domain of this thesis, the distributions are separated, the ratio is extreme, and the clip saturates. The practical consequence is a clip fraction of 0.000: the imitation gradient is zero at every training iteration. This is not a flaw in the original formulation but a consequence of the domain difference. The same mechanism that stabilises imitation in the original setting removes it here.

On [34], the scale sweep provides direct evidence that the relationship is not simply one of batch size. An eightfold increase in the parallel batch leaves the self-play held-out result at zero. The success of self-play at the scale of Berner et al. rested on properties of the domain and the action space that the present study varies one at a time.

6.3 Contributions Revisited

The three contributions of Section 1 can now be given quantitative content.

The primary contribution is the mechanism behind the self-play result. The within-distribution evaluation shows the self-play policy solves 61.4 % of the disc goals and 11.7 % of the T-block goals the goal-proposing agent itself produces (5-seed means), against 7.3 % and 0.0 % of held-out scenes. The mechanism is therefore two compounding effects: the coupled multi-objective gate bounds competence on the proposer's own goals, and the narrow goal distribution prevents that competence from transferring. The single-agent models show no such gap, because their training goals cover the suite. Five measurements support the diagnosis: the disc contrast (Section 5.3.1), the within-distribution evaluation (Section 5.4), the scale-sweep null result (Section 5.3.3), the ABC clip-saturation measurement (Section 5.3.4), and the goal-encoder ablation (Section 5.3.5).

The second contribution is the setup contrast. A structured comparison isolates each of five dimensions on which this task differs from the settings where self-play succeeded: goal coupling, reward formulation, imitation signal, goal representation, and compute scale. The coupled gate is the dimension that matters: it reduces the within-distribution rate from the disc's 61.4 % to the T-block's 11.7 %, while the other dimensions contribute nothing measurable.

The third contribution is the established task solvability. The single-agent models solve 79–80 % of held-out scenes with tight confidence intervals across five seeds, which anchors the self-play result as a property of the method rather than the task. The curriculum model is equivalent to

the single-agent model at the large Isaac Lab batch, which refines the surveys of [7] and [25]: a well-shaped dense reward and a curriculum are partial substitutes whose point of equivalence depends on the per-update batch size.

6.4 Limitations

Several limitations bound the scope of these conclusions. All held-out numbers in this thesis come from the full evaluation protocol of Section 4.3, across every trained seed; no preliminary pass is reported anywhere. The within-distribution and held-out evaluations use the same scene protocol, so the comparison of Section 5.4 is not confounded by budget; the earlier per-push training measure is retained only as an auxiliary diagnostic. The comparison between self-play and single-agent training differs on more than one dimension at once: the number of agents, the goal distribution, the episode budget, the goal encoder, and the behavioural-cloning term. The single-dimension isolations of Sections 5.3.1 to 5.3.5 provide the decomposition. The dimensions of variation are enumerated in Appendix G.

The improvement-reward baseline (PPO-Baseline) is still training at the time of writing, so its held-out validation is pending and the reward-efficiency comparison against the potential-based agent remains incomplete. This is the only measurement in this thesis still in progress. The sensitivity of the potential-based reward to its parameters was tested only in a coarse single-seed grid (Appendix C). The off-policy SAC baseline did not converge within the available budget.

6.5 Future Work

Each limitation suggests a concrete continuation. The one measurement still in progress, the improvement-reward baseline, will complete the reward-efficiency comparison once its held-out validation finishes.

Beyond these pending measurements, three directions follow from the mechanism. The first is a goal-proposing agent that samples goals from a wider distribution than its own reachable region, for instance by proposing goals relative to the object's start pose rather than its own final pose, or by perturbing proposed goals away from the demonstrator's reachable region. If the generalisation gap closes under such a proposer, the diagnosis of Section 5.4 is confirmed directly. The second direction is an unclipped or adaptively clipped cloning loss, or one defined on the decoded continuous push parameters rather than the discrete bins. The clip fraction of zero establishes that the current formulation contributes nothing in this action space; if the original formulation can be adapted to the discrete domain, the demonstration signal may be restored. The third direction concerns the coupled gate itself. A relaxed or partial-credit gate that rewards satisfying the position and orientation gates independently, rather than only their conjunction, would test whether the disc's 61.4 % within-distribution competence can be preserved on the T-block.

The practical recommendation that follows from the full set of results is unchanged in its direction and refined in its mechanism. For a contact-rich multi-objective goal-reaching task of this kind, a principled dense reward is the first investment. It produces a single-agent policy that generalises to held-out scenes. Structured training dynamics, whether a curriculum or self-play, should be added only in the small-batch setup where they demonstrably contribute. Asymmetric self-play, in the form tested here, derives its curriculum from its own reachable goal distribution, and that distribution does not cover the evaluation. The failure is one of gen-

eralisation compounded by the coupled gate, which bounds the competence the policy acquires on its own goals.

6.6 Conclusion

This thesis compared reward shaping against structured training dynamics on a single contact-rich planar-pushing task, under a common budget and a common success criterion. Potential-based reward shaping produced a single-agent policy that solves 79 % of held-out scenes with confidence intervals across five seeds, which anchors the task as solvable. Asymmetric self-play solved none. The contrast between the two setups was decomposed across five dimensions: the coupled multi-objective gate emerged as one of two compounding factors, while compute scale, the imitation signal, and the goal representation each contributed nothing. The mechanism is a generalisation failure compounded by the coupled gate. On the goal distribution its own curriculum produces, the self-play policy solves 61.4 % of disc scenes and 11.7 % of T-block scenes, and transfers none of it to the held-out suite. This is the regime in which a method that succeeds elsewhere, in reversible domains and at distributed scale, underperforms a plain shaped single-agent policy here.

Bibliography

[1] M. Q. Mohammed, L. C. Kwek, S. C. Chua, A. Al-Dhaqm, S. Nahavandi, T. A. E. Eisa, M. F. Miskon, M. N. Al-Mhiqani, A. Ali, M. Abaker, and E. A. Alandoli, "Review of learning-based robotic manipulation in cluttered environments," *Sensors*, vol. 22, no. 20, p. 7938, 2022.

[2] A. Zeng, S. Song, S. Welker, J. Lee, A. Rodriguez, and T. Funkhouser, "Learning synergies between pushing and grasping with self-supervised deep reinforcement learning," in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 4238–4245, IEEE, 2018.

[3] K. Mokhtar, C. J. M. Heemskerk, and H. Kasaei, "Self-supervised learning for joint pushing and grasping policies in highly cluttered environments," *ArXiv*, vol. abs/2203.02511, 2022.

[4] H. Kasaei and M. M. Kasaei, "Harnessing the synergy between pushing, grasping, and throwing to enhance object manipulation in cluttered scenarios," 2024.

[5] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *ArXiv*, vol. abs/1707.06347, 2017.

[6] A. Y. Ng, D. Harada, and S. Russell, "Policy invariance under reward transformations: Theory and application to reward shaping," in *Proceedings of the Sixteenth International Conference on Machine Learning (ICML)*, pp. 278–287, 1999.

[7] S. Narvekar, B. Peng, M. Leonetti, J. Sinapov, M. E. Taylor, and P. Stone, "Curriculum learning for reinforcement learning domains: A framework and survey," *arXiv preprint arXiv:2003.04960*, 2020.

[8] S. Sukhbaatar, Z. Lin, I. Kostrikov, G. Synnaeve, A. Szlam, and R. Fergus, "Intrinsic motivation and automatic curricula via asymmetric self-play," in *International Conference on Learning Representations (ICLR)*, 2018.

[9] OpenAI, M. Plappert, R. Sampedro, T. Xu, I. Akkaya, V. Kosaraju, P. Welinder, R. D'Sa, A. Petron, H. P. de Oliveira Pinto, A. Paino, H. Noh, L. Weng, Q. Yuan, C. Chu, and W. Zaremba, "Asymmetric self-play for automatic goal discovery in robotic manipulation," *ArXiv*, vol. abs/2101.04882, 2021.

[10] M. T. Mason, "Mechanics and planning of manipulator pushing operations," *The International Journal of Robotics Research*, vol. 5, no. 3, pp. 53–71, 1986.

[11] S. Goyal, A. Ruina, and J. Papadopoulos, "Planar sliding with dry friction part 1. limit surface and moment function," *Wear*, vol. 143, no. 2, pp. 307–330, 1991.

[12] M. Mittal, C. Yu, Q. Yu, J. Liu, N. Rudin, D. Hoeller, J. L. Yuan, R. Singh, Y. Guo, H. Mazhar, A. Mandlekar, B. Babich, G. State, M. Hutter, and A. Garg, "Orbit: A unified simulation framework for interactive robot learning environments," *IEEE Robotics and Automation Letters*, vol. 8, no. 6, pp. 3740–3747, 2023.

[13] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. The MIT Press, second ed., 1998.

[14] M. Yang, Y. Lin, A. Church, J. Lloyd, D. Zhang, D. A. W. Barton, and N. F. Lepora, "Sim-to-real model-based and model-free deep reinforcement learning for tactile pushing," *IEEE Robotics and Automation Letters*, vol. 8, pp. 5480–5487, 2023.

[15] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. A. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, pp. 529–533, 2015.

[16] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," *ArXiv*, vol. abs/1801.01290, 2018.

[17] J. Schulman, S. Levine, P. Abbeel, M. I. Jordan, and P. Moritz, "Trust region policy optimization," *ArXiv*, vol. abs/1502.05477, 2015.

[18] M. Andrychowicz, F. Wolski, A. Ray, J. Schneider, R. Fong, P. Welinder, B. McGrew, J. Tobin, O. Abbeel, and W. Zaremba, "Hindsight experience replay," *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[19] M. Grześ, "Reward shaping in episodic reinforcement learning," in *Proceedings of the 16th Conference on Autonomous Agents and MultiAgent Systems (AAMAS)*, pp. 565–573, 2017.

[20] S. Devlin and D. Kudenko, "Dynamic potential-based reward shaping," in *Proceedings of the 11th International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, pp. 433–440, 2012.

[21] A. Harutyunyan, S. Devlin, P. Vrancx, and A. Nowé, "Expressing arbitrary reward functions as potential-based advice," in *Proceedings of the Twenty-Ninth AAAI Conference on Artificial Intelligence*, pp. 2652–2658, 2015.

[22] M. Grześ and D. Kudenko, "Theoretical and empirical analysis of reward shaping in reinforcement learning," in *2009 International Conference on Machine Learning and Applications (ICMLA)*, pp. 337–344, IEEE, 2009.

[23] C. Florensa, D. Held, M. Wulfmeier, M. Zhang, and P. Abbeel, "Reverse curriculum generation for reinforcement learning," in *Conference on Robot Learning (CoRL)*, pp. 482–495, PMLR, 2017.

[24] S. Luo, H. Kasaei, and L. Schomaker, "Accelerating reinforcement learning for reaching using continuous curriculum learning," *2020 International Joint Conference on Neural Networks (IJCNN)*, pp. 1–8, 2020.

[25] R. Portelas, C. Colas, L. Weng, K. Hofmann, and P.-Y. Oudeyer, "Automatic curriculum learning for deep rl: A short survey," *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence (IJCAI)*, pp. 4819–4825, 2020.

[26] F. Torabi, G. Warnell, and P. Stone, "Behavioral cloning from observation," in *Proceedings of the Twenty-Seventh International Joint Conference on Artificial Intelligence (IJCAI)*, pp. 4950–4957, 2018.

[27] P. Florence, C. Lynch, A. Zeng, O. A. Ramirez, A. Wahid, L. Downs, A. Wong, J. Lee, I. Mordatch, and J. Tompson, "Implicit behavioral cloning," in *Conference on Robot Learning (CoRL)*, pp. 158–168, PMLR, 2022.

[28] T. Hester, M. Vecerík, O. Pietquin, M. Lanctot, T. Schaul, B. Piot, D. Horgan, J. Quan, A. Sendonaris, I. Osband, G. Dulac-Arnold, J. P. Agapiou, J. Z. Leibo, and A. Gruslys, "Deep q-learning from demonstrations," in *AAAI Conference on Artificial Intelligence*, 2017.

[29] T. Silver, K. Allen, J. Tenenbaum, and L. Kaelbling, "Residual policy learning," *arXiv preprint arXiv:1812.06298*, 2018.

[30] M. Dennis, N. Jaques, E. Vinitsky, A. Bayen, S. Russell, A. Critch, and S. Levine, "Emergent complexity and zero-shot transfer via unsupervised environment design," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 13049–13061, 2020.

[31] A. Campero, R. Raileanu, H. Küttler, J. B. Tenenbaum, T. Rocktäschel, and E. Grefenstette, "Learning with amigo: Adversarially motivated intrinsic goals," in *International Conference on Learning Representations (ICLR)*, 2021.

[32] I. Durugkar, M. Tec, S. Niekum, and P. Stone, "Adversarial intrinsic motivation for reinforcement learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, pp. 8622–8636, 2021.

[33] A. Letcher, D. Balduzzi, S. Racanière, J. Martens, J. Foerster, K. Tuyls, and T. Graepel, "Differentiable game mechanics," *Journal of Machine Learning Research*, vol. 20, no. 84, pp. 1–40, 2019.

[34] C. Berner, G. Brockman, B. Chan, V. Cheung, P. Debiak, C. Dennison, D. Farhi, Q. Fischer, S. Hashme, C. Hesse, *et al.*, "Dota 2 with large scale deep reinforcement learning," *arXiv preprint arXiv:1912.06680*, 2019.

[35] S. Sukhbaatar, E. Denton, A. Szlam, and R. Fergus, "Learning goal embeddings via self-play for hierarchical reinforcement learning," in *arXiv preprint arXiv:1811.09083*, 2018.

[36] N. Tishby and N. Zaslavsky, "Deep learning and the information bottleneck principle," in *2015 IEEE Information Theory Workshop (ITW)*, pp. 1–5, IEEE, 2015.

[37] A. Goyal, R. Islam, D. Strouse, Z. Ahmed, H. Larochelle, M. Botvinick, S. Levine, and Y. Bengio, "Infobot: Transfer and exploration via the information bottleneck," in *International Conference on Learning Representations (ICLR)*, 2019.

[38] O. Nachum, S. S. Gu, H. Lee, and S. Levine, "Data-efficient hierarchical reinforcement learning," in *Neural Information Processing Systems*, 2018.

[39] A. S. Vezhnevets, S. Osindero, T. Schaul, N. Heess, M. Jaderberg, D. Silver, and K. Kavukcuoglu, "Feudal networks for hierarchical reinforcement learning," in *International Conference on Machine Learning (ICML)*, pp. 3540–3549, PMLR, 2017.

[40] P.-L. Bacon, J. Harb, and D. Precup, "The option-critic architecture," in *Proceedings of the Thirty-First AAAI Conference on Artificial Intelligence*, pp. 1726–1734, 2017.

[41] M. Hutsebaut-Buysse, K. Mets, and S. Latré, "Hierarchical reinforcement learning: A survey and open research challenges," *Machine Learning and Knowledge Extraction*, vol. 4, no. 1, pp. 172–221, 2022.

[42] A. V. Nair, V. Pong, M. Dalal, S. Bahl, S. Lin, and S. Levine, "Visual reinforcement learning with imagined goals," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 31, 2018.

[43] K. M. Lynch and M. T. Mason, "Stable pushing: Mechanics, controllability, and planning," *The International Journal of Robotics Research*, vol. 15, no. 6, pp. 533–556, 1996.

[44] R. D. Howe and M. R. Cutkosky, "Practical force-motion models for sliding manipulation," *The International Journal of Robotics Research*, vol. 15, no. 6, pp. 557–572, 1996.

[45] J. Stüber, C. Zito, and R. Stolkin, "Let's push things forward: A survey on robot pushing," *Frontiers in Robotics and AI*, vol. 7, p. 8, 2020.

[46] K.-T. Yu, M. Bauza, N. Fazeli, and A. Rodriguez, "More than a million ways to be pushed: A high-fidelity experimental dataset of planar pushing," *2016 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 30–37, 2016.

[47] F. C. Park, J. E. Bobrow, and S. R. Ploen, "A lie group formulation of robot dynamics," *The International Journal of Robotics Research*, vol. 14, no. 6, pp. 609–618, 1995.

[48] J. Urain, N. Funk, J. Peters, and G. Chalvatzaki, "Se(3)-diffusionfields: Learning smooth cost functions for joint grasp and motion optimization through diffusion," in *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 5923–5930, IEEE, 2023.

[49] E. van der Pol, D. E. Worrall, H. van Hoof, F. A. Oliehoek, and M. Welling, "Mdp homomorphic networks: Group symmetries in reinforcement learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 4199–4210, 2020.

[50] H. Huang, D. Wang, R. Walters, and R. Platt, "Equivariant transporter network," in *Robotics: Science and Systems (RSS)*, 2022.

[51] H. Nguyen, A. Baisero, D. Klee, D. Wang, R. Platt, and C. Amato, "Equivariant reinforcement learning under partial observability," in *Conference on Robot Learning (CoRL)*, 2024.

[52] V. Makoviychuk, L. Wawrzyniak, Y. Guo, M. Lu, K. Storey, M. Macklin, D. Hoeller, N. Rudin, A. Allshire, A. Handa, and G. State, "Isaac gym: High performance gpu-based physics simulation for robot learning," in *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2021.

[53] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger, "Deep reinforcement learning that matters," in *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence*, pp. 3207–3214, 2018.

[54] R. Agarwal, M. Schwarzer, P. S. Castro, A. C. Courville, and M. G. Bellemare, "Deep reinforcement learning at the edge of the statistical precipice," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, pp. 29304–29320, 2021.

Appendices

A Push Mechanics: Limit Surface and Characteristic Length

The limit-surface normality condition of Equation 14 states that the instantaneous motion twist of a quasi-statically pushed object is normal to its friction limit surface at the point corresponding to the applied wrench. Its practical consequence for the task concerns any off-centre push, one whose line of action does not pass through the object's centre of friction. Such a push produces a wrench with a non-zero moment component, and therefore a twist that contains an angular velocity. Almost every push consequently changes both position and orientation. The characteristic length L used in the SE(2) distance of Equation 20 is the radius of gyration of the object, a physical constant; for the T-block $L = 0.07\,\mathrm{m}$. It is not a tuning parameter, and it sets the weight of an orientation error relative to a position error in a manner consistent with the object's mass distribution. For the rotationally symmetric disc the characteristic length is set to zero, which collapses the SE(2) distance to a pure position distance and encodes the absence of an orientation objective. The mechanical contrast between the two objects, the coupled twist of the T-block against the pure translation of the disc, is discussed in Section 2.8.

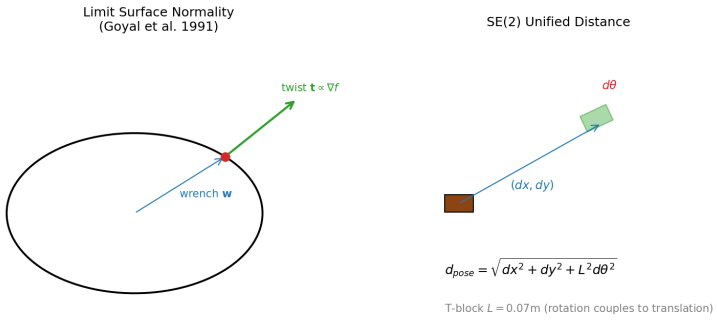

Figure 16: An off-centre push produces coupled translation and rotation of the T-block. The motion twist is normal to the friction limit surface.

B Training Curves

Figure 17 shows the training-time success rate and value loss for the single-agent and curriculum models. The single-agent model improves steadily over roughly 3,000 iterations, the curriculum model follows a similar trajectory, and the value loss remains stable throughout, which reflects the bounded range of the exponential potentials.

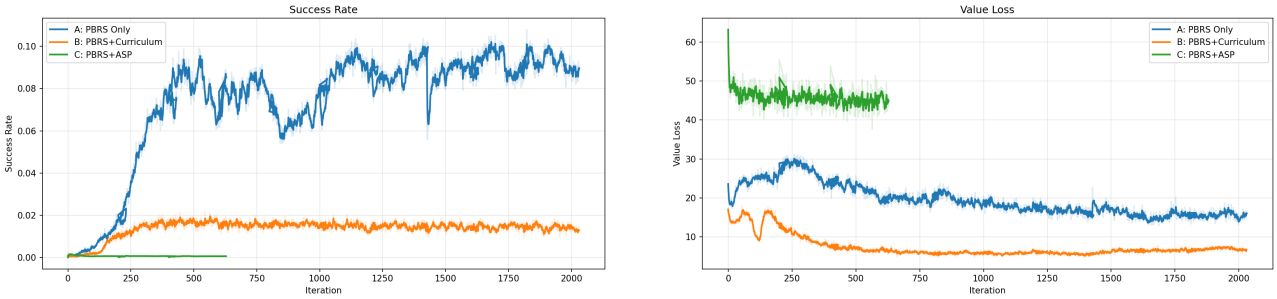

Figure 17: Training success rate (left) and value loss (right) for the potential-based models.

C PBRS Parameter Sensitivity

The potential-based reward uses $k_p = 30$, $k_r = 5$, and $w_{\text{pos}} = w_{\text{rot}} = 10$, chosen from pilot runs. A systematic grid over these parameters was not performed within the available budget. The robustness of the reward to its parameters is therefore not established here. It is recorded as a limitation in Section 6.4 and as a direction for future work in Section 6.5. A grid over $k_p \in \{10, 20, 30, 50\}$ and $w \in \{5, 10, 20\}$, evaluated at the 30-scene gate, would provide the missing sensitivity analysis.

D Validation Scene Descriptions

The 30-scene held-out suite is defined in `validation_configs.py` and comprises three groups of ten. Scenes 1 to 10 are rotation-dominant, with orientation changes up to π radians and mixed translations. Scenes 11 to 20 are position-only, with the goal orientation equal to the start orientation. Scenes 21 to 30 combine a position change with an orientation change. Within each group, difficulty is graded easy, medium, and hard by the start-to-goal distance and the magnitude of the required rotation. All coordinates are expressed in the environment-local frame in metres, and orientations in radians.

E cuRobo IK Pipeline and Workspace

Each decoded push primitive is executed as a five-phase Cartesian trajectory, approach, descend, push, retract, and return, spanning 72 physics substeps. Every waypoint is mapped to arm joint targets by the cuRobo 0.7.5 batched Inverse Kinematics solver, with the gripper held closed throughout. The reachable workspace is bounded to the region over the table where the solver returns elbow-up solutions. Pushes whose start or end points fall outside this region are clamped to the boundary before execution, so the credit assigned to an action matches the motion actually performed.

F Observation Space Specifications

The single-agent models use a 28-dimensional observation: end-effector pose (6), object pose and velocity (14), goal pose (6), and the position and orientation errors (2). The goal-solving agent in the self-play models uses the same 28-dimensional layout, with the goal encoded through the GoalEncoder into an eight-dimensional latent in the difference variant. The goal-proposing agent uses a 20-dimensional observation that omits the goal, since it sets the goal rather than solving it.

G Confounded Ablation Disaggregation

The single-agent and self-play models differ on at least eight dimensions at once: the number of agents (one against two), the goal distribution (fixed random against adversarial and shifting), the goal encoder (absent against present), the behavioural-cloning term (absent against present), the historical opponent pool (absent against present), the rollout length (15 pushes for one agent, against 5 plus 10 split between two), the episode budget (5 pushes for the single agent against 10 for the goal-solver), and the number of policy updates per iteration (one against two). The comparison in this thesis therefore isolates self-play as a package rather than any single mechanism within it. Four single-dimension isolations are provided nonetheless: the disc contrast of Section 5.3.1, the two Alice reward formulations of Section 5.3.2, the imitation ablations of Section 5.3.4, and the goal-encoder ablation of Section 5.3.5. The cleanest of these is the reward-formulation pair, whose members differ only in the goal-proposing agent's reward. It shows no consistent advantage for either formulation. At 528 environments the outcome-based variant logs a per-push training rate of 35.8 % (± 8.8 pp, 5 seeds) against 32.6 % (± 0.4 pp, 5 seeds) for the time-based variant on the T-block, and both transfer zero held-out scenes. On the disc the time-based variant leads during training (83.8 % against 77.9 % per push, 5 seeds each) yet trails slightly on held-out scenes (6.7 % against 7.3 %).

H Training Budget Comparison

Every Isaac Lab run in the experiment set is budget-matched. The total experience is held at about 23.8 million pushes, and the iteration count is set inversely proportional to the number of parallel environments. A sweep therefore changes the per-update batch size alone. The confound that a larger batch also means fewer gradient updates is stated wherever a scale result appears. The headline models train at 528 environments for approximately 3,000 iterations. The potential-based agent and the improvement-reward baseline are additionally swept across 256 to 2,048 environments, and the two self-play subjects across 256 and 2,048. At the 528-environment baseline the potential-based single-agent model reaches a held-out scene success rate of 79.3 % (± 3.5 pp, 5 seeds), and the curriculum model 80.0 % (± 4.1 pp, 5 seeds). The improvement-reward baseline, trained at up to 2,048 environments, is still training at the time of writing; its held-out validation is **TBD**, and until it completes the environment-efficiency comparison between the two reward families remains open.

I Secondary Variants

The two disc self-play models and the Bob-penalty model are headline subjects, reported in Table 4 and Sections 5.2 and 5.3.1; they are not repeated here. The ablation arms, with the imitation term removed, with the goal encoder removed, and the disc positive control, appear in Sections 5.3.4 and 5.3.5. Two groups remain outside the main text. The first is the original, non-simplified potential formulation at the position-only gate. The second is the cross-environment set: the single-agent, curriculum, and self-play models retrained in `gym-pusht`, one seed each, which solve 3.3 %, 9.0 %, and 1.0 % of the held-out scenes respectively. Both groups are consistent with the principal findings: self-play remains far below single-agent training across every variant.